

응용 · CODEX CLI · VS CODE 강의 교안

Coding Agent 실전

AI기반 TDD 마스터 과정

개념에서 코드까지, 추적성을 잃지 않는 개발 (Concept-to-Code)

4계층 검증 구조

계약 ID

추적성 매트릭스

Dual-Track TDD $\equiv$ ARRR

하네스 엔지니어링

하루 8시간 집중 과정 · 08:30-17:30 | 강의 40% + 실습 60% | 개발자 및 입문자 대상

2026 · 김대경 Ph.D.

과정 개요 — 하루 만에 C2C 개발 흐름을 체득한다

항목	내용
교육 대상	개발자 및 입문자 (Git 기초 숙지 전제)
교육 시간	8시간 — 08:30~17:30 (점심 11:45~13:00) · 0장(사전 과제)은 시간에서 제외
교육 방식	강의 40% + 실습 60% (개념 → 시연 → 개별 실습)
도구 스택	VS Code · Codex CLI(터미널 AI 에이전트) · Codex IDE 확장(VS Code)
사전 과제	0장(설치·CLI 기본기)은 교육 전 개별 완료 — 당일엔 체크리스트만 확인
관통 예제	장바구니 할인 계산기 — 계약 ID 6개(INV-1~4, E-1~2)를 ARRR로 완성
적용 과제	Magic Square 4×4 관통 프로젝트(8장) · 부록 A 적용 사례(제조 RAG · 모바일 MVP)
심화 옵션	MCP 외부 도구 연동(Context7 · Sequential Thinking · GitHub) · Hooks — 부록 C

목표

핵심 목표 — 개념이 코드까지 추적되는(C2C) 개발 흐름을 1일 안에 체득한다. 오늘의 산출물: docs/PRD.md · tests/ · src/ · docs/traceability.md

목차 — 이론 · 실습 · 적용 과제

PART I 이론	PART II 실습 — 장바구니 할인 계산기	PART III 적용 과제 · 마무리
0장 사전 과제 — 환경 준비	4장 실습 준비와 하네스 엔지니어링	8장 Magic Square 4×4 — 관통 프로젝트
1장 개념과 원리 — 바이브 코딩을 넘어서	5장 제품 계약 — 상류(Discovery)	9장 마무리 — Review · KPT 회고
2장 단위 테스트 설계	6장 Track B — Entity/Logic	부록 A 적용 사례 · B 하네스 이식 · C MCP·Hooks
3장 클린 코드와 리팩토링	7장 Track A — UI/Boundary	

🕒 08:30 0장 확인 → 1~3장 이론 → 11:45 점심 → 4~7장 실습 → 8장 적용 → 9장 회고 (17:30)



PART I

이론 — 왜, 무엇을, 어떻게 검증하는가

환경 준비부터 TDD·C2C 원리, 단위 테스트 설계, 클린 코드와 리팩토링까지 — 실습에 들어가기 전에 갖춰야 할 개념의 뼈대를 세웁니다.

00



CHAPTER 0

사전 과제 — 환경 준비

설치와 CLI 기본기는 교육 전에 개별 완료합니다. 당일 0교시에는 체크리스트만 5분 확인하고 바로 1장(이론)으로 진입합니다.

🕒 사전 과제 · 강의 시간에서 제외

준비물 점검 — 시작 전에 갖춰야 할 것

환경 — 버전 확인

```
# 아래 4줄이 모두 통과해야 시작
$ node --version    # v20.x 이상
$ python --version  # 3.11.x 이상
$ git --version     # 2.x
$ codex --version   # 없으면 0.2에서 설치
```

PC 개인 PC

Windows · macOS · Linux (듀얼 모니터 권장)

ID GitHub · ChatGPT 계정

ChatGPT 계정(Plus 이상) 또는 API 키로 Codex CLI·IDE 확장 로그인

SW Node.js 20+ · Python 3.11+ · Git

CLI 설치와 pytest·Flask 실습에 사용

주의

Node.js·npm이 없으면 `npm install -g @openai/codex` 설치가 실패합니다 — 가장 흔한 사전 준비 실패 원인이니 반드시 먼저 확인하세요.

도구 스택과 설치 — 세 가지의 역할 분담

1 VS Code

에디터 + 통합 터미널 — 코드 열람·수동 편집, 터미널에서 codex·pytest 실행

2 Codex CLI

터미널 AI 에이전트 — 다중 파일 편집·테스트 자동 실행·자율 반복 (ARRR 주력)

3 Codex IDE 확장

VS Code 확장(채팅·Agent 모드) — 코드 질의·인라인 편집, Agent 모드는 CLI와 같은 엔진

```
# VS Code 확장 마켓플레이스에서 "Codex" (OpenAI) 설치 후, 터미널에서:
$ npm install -g @openai/codex
$ codex          # 첫 실행 시 ChatGPT 계정(또는 API 키) 로그인 → 대화형 세션
> /model         # 기본 권장 모델 유지 — 복잡한 추론은 추론 강도(reasoning)를 high로
```

핵심

CLI와 IDE 확장은 같은 작업의 두 얼굴 — 교안의 ► 프롬프트 박스는 어느 쪽에 넣어도 동일하게 동작합니다.

Codex CLI 기본기 — 핵심 진입점 5가지

진입점	무엇 · 언제	예
<code>codex</code>	대화형 세션 — 자연어로 지시 (모든 ARRR 작업)	<code>codex</code>
<code>codex exec "..."</code>	단발 실행 후 종료 — 스크립트·파이프라인용	<code>codex exec "pytest 실패 요약"</code>
@경로	파일/폴더를 컨텍스트에 정확히 지정	@src/cart.py 설명해줘
!명령	세션 안에서 셸 명령 직접 실행	!pytest -q
/명령 · \$스킬	내장 슬래시 명령(/model · /permissions · /plan) · 커스텀 스킬 호출 (\$이름)	\$tdd-red

권한 설정 (-s / -a)	-s read-only	-a untrusted	Auto (기본)	--yolo
동작	읽기 전용 — 분석·계획만	변경 명령마다 승인 (권장 시 작점)	작업 폴더 안 편집·실행 자동 승인	샌드박스·승인 해제 (격리 저장소에서만)

주의

처음에는 반드시 -a untrusted로 시작해 변경마다 직접 승인하세요. --yolo는 되돌릴 수 있는 격리된 실습 저장소에서만 사용합니다.

01



CHAPTER 1

개념과 원리 — 바이트 코딩을 넘어서

왜 테스트를 먼저 두는가? Dual-Track TDD(RED → GREEN)와 C2C 4계층 검증 구조로 추적 가능한 개발의 원리를 세웁니다. Codex CLI · Python 3.11+ · pytest · Git

🕒 1교시 · 이론

문제의식 — 바이브 코딩과 C2C 단절

바이브 코딩 (Vibe Coding)

AI에게 즉흥적 요청을 반복해 기능을 얻어내는 방식. 동작은 하지만 “이 코드가 어느 요구·개념에서 나왔는가”를 역추적할 수 없다 = C2C 단절.

그래서 생기는 일

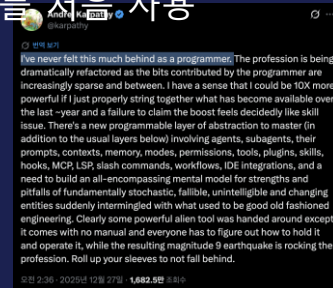
- 요구 만족의 근거가 없다
- 무엇을 왜 고쳤는지 나중에 추적 불가
- 에이전트가 오래 일할수록 “통과한 듯 보이는” 코드가 는다

체험

미니 체험 — Codex에게 “계산기 만들어줘” 한 줄만 던져 보고, 결과 코드의 각 줄이 어떤 요구에서 나왔는지 설명할 수 있는지 확인해 봅니다.

탄생과 확산 (2025년 초)

- 2025년 2월, OpenAI 공동 창립자 안드레이 카파시(Andrej Karpathy)가 ‘바이브 코딩’이라는 용어를 처음 사용



- 개발자가 직접 코드를 짜는 ‘구현(How)’ 중심에서 AI에게 방향을 지시하는 ‘의도(What)’ 중심으로 패러다임 전환

발전과 한계 (2025년 말 ~ 2026년)

- 생성된 코드를 사람이 온전히 이해하지 못해 운영 환경에서 보안 결함·유지보수 문제가 드러남
- AI 시대에 인간 중심 언어가 필요한가, 기계 친화적 언어를 둘 것 인가에 대한 새 논쟁

개발의 다섯 단계 — 리서치에서 디버깅까지

1

리서치

요구·기존 코드·제약
을 파악한다

2

계획

접근 전략과 작업 순
서를 설계한다

3

코드

계획을 실제 구현으
로 옮긴다

4

테스트

의도대로 동작하는지
검증한다

5

디버깅

실패 원인을 추적하
고 고친다

핵심

에이전트와 함께하는 개발도 이 다섯 단계를 지냅니다 — 관건은 각 단계를 계약과 커밋으로 “추적 가능”하게 남기는 것(C2C). 바이트 코딩은 이 흐름을 “한 줄 요청”으로 뭉칩니다.

왜 “테스트 먼저”인가

판단의 조급성 · Discovery

문서는 바람(needs)의 나열이었을 뿐, 서로 합의하고 증거로 판정할 수 있는 가설·수용조건이 아니다.

행동의 조급성 · Delivery

검증 근거 없이 구현이 앞서면, 통과한 것처럼 보이지만 실제로는 아닌 코드가 쌓인다.

[시스템 '개발' 현장의 모습]



[시스템 '개발' 운영의 모습]



문제

미리 판단한 상태에서 개발이 끝나갈 무렵에야 “이게 요구사항이었나?”라는 질문이 테스트 시점에 터진다 — 질문이 판단·행동을 앞서지 못하고 맨 뒤로 밀려난 전형이다.

빠르게만 만들면 생기는 문제들

1 검증 근거 부재

이 코드가 정말 요구사항을 만족하는지 객관적으로 보여 줄 근거가 없다

2 추적 어려움

어떤 변경이 무엇을 왜 고쳤는지 나중에 따라가기 어렵다

3 위장된 통과

세션이 길어질수록 통과한 것처럼 보이지만 실제로는 아닌 코드가 는다

전통적 방식

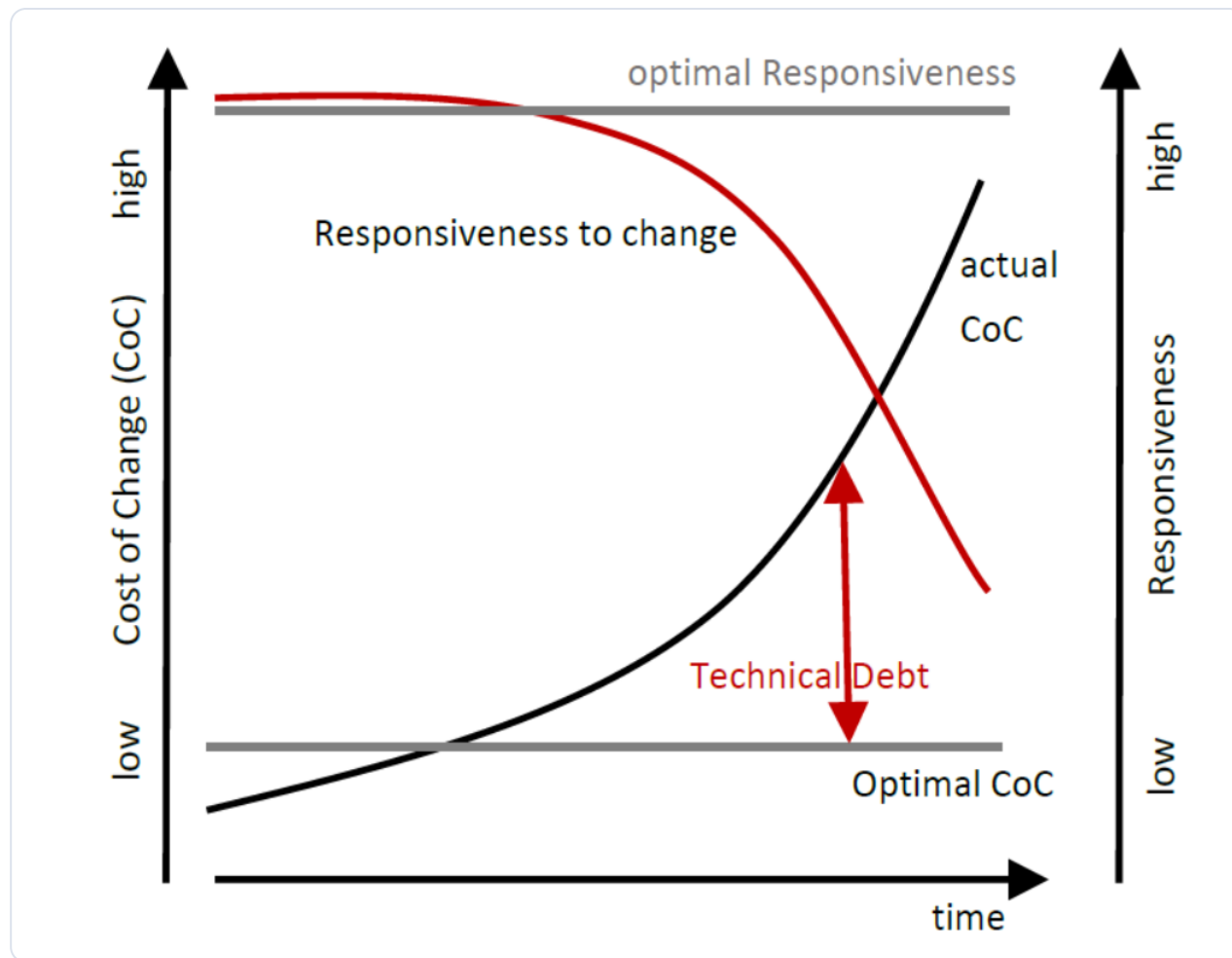
기획서 → 개발자 해석 → 구현 → (뒤늦게) 문제 발견



Dual-Track TDD

실패 조건 합의 → RED 테스트 → 최소 구현(GREEN)

클린 코드와 변경 비용 — 시간이 지날수록



시간이 지날수록	클린 코드 X	클린 코드 ✓
변경 비용	기하급수적 증가	완만하게 유지
응답 속도	점점 느려짐	빠른 응대 유지
버그 발생	수정 시 부작용	테스트로 즉시 감지

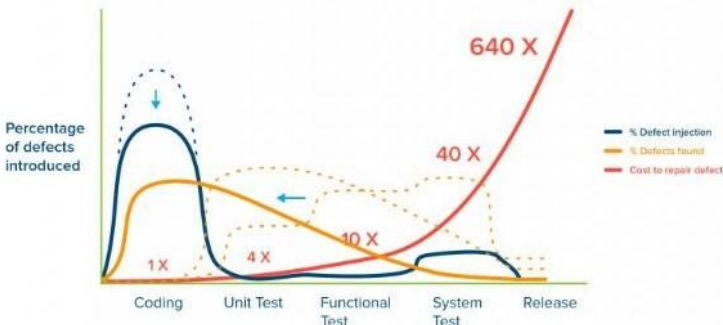
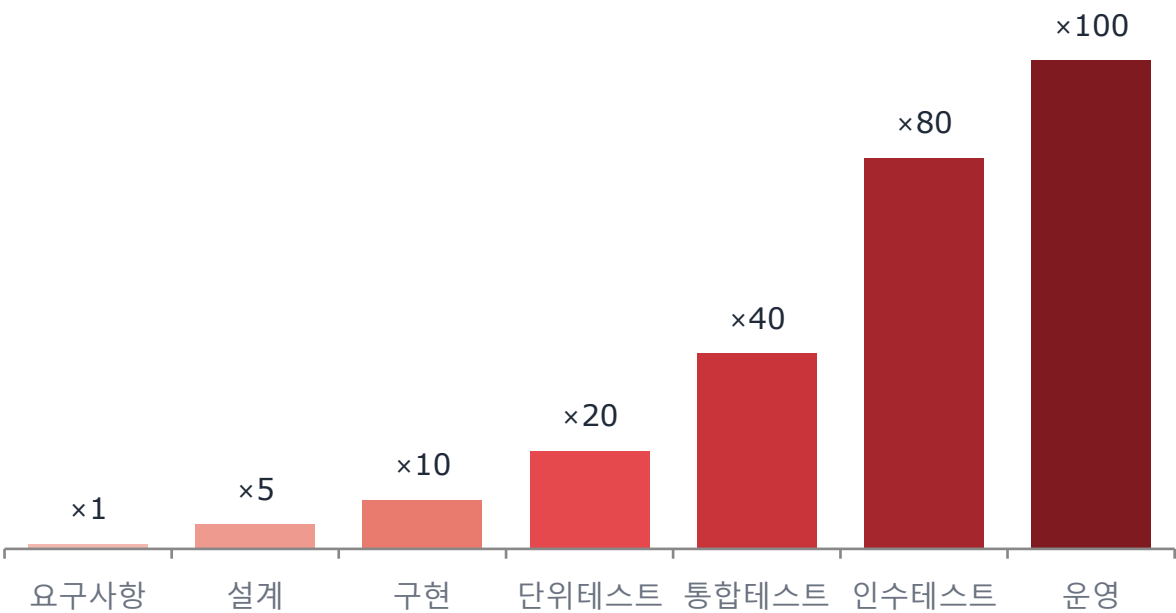
그래프 읽는 법

- 빨간 곡선: 변화에 대한 대응력(Responsiveness)이 시간이 갈수록 떨어진다
- 검은 곡선: 실제 변경 비용(CoC)이 기하급수적으로 오른다
- 두 곡선 사이의 간격 = 기술 부채(Technical Debt)

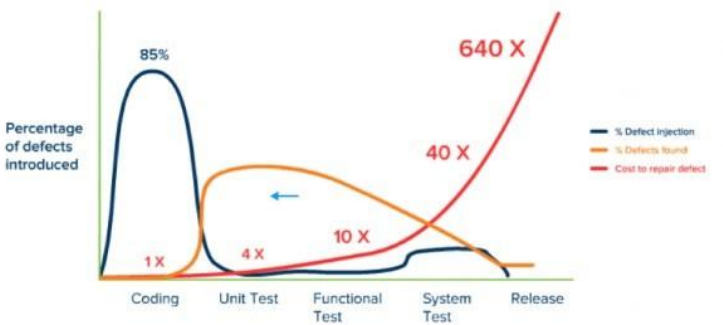
테스팅의 비용 — 결함은 늦게 발견할수록 비싸다

결함은 늦게 발견할수록 수정 비용이 기하급수적으로 증가한다 (Cost of Defect).

단계별 결함 수정 비용 (상대값)



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.

근거 TDD: 개발 시간 15~35% 증가 → 결함률 40~90% 감소 (Microsoft·IBM 연구)

TDD 한 사이클 — RED · GREEN · REFACTOR (Python)

RED

실패하는 테스트를 먼저

만지는 것

tests/ 만 — 구현(src/)은 건드리지 않는다

끝나는 신호

테스트가 “의도대로” 실패

GREEN

최소 구현으로 통과

만지는 것

src/ 만 — 요구사항에 없는 기능은 금지

끝나는 신호

테스트가 전부 통과

REFACTOR

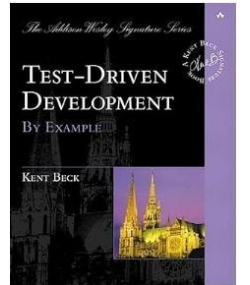
동작 그대로 구조 정리

만지는 것

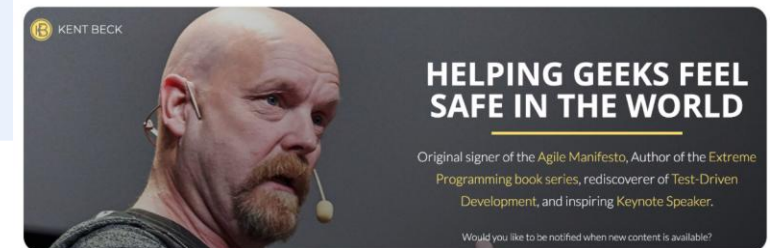
중복 제거·이름 정리 — 계약 불변

끝나는 신호

여전히 GREEN 유지



Kent Beck,
Test-Driven
Development



(출처: kentbeck.com)

```
# RED — tests/test_calc.py
def test_add(): assert add(2, 3) == 5
# GREEN — src/calc.py
def add(a, b): return a + b
```

같은 기능을 TDD로 재구현 — Codex CLI 세션에서

RED — 의도된 실패

▶ 프롬프트

tests/test_calc.py 에 test_add 테스트만 작성해줘. src/calc.py 는 만들지 마.

결과 add가 없으니 pytest는 의도된 실패 (RED).

GREEN — 최소 통과

▶ 프롬프트

방금 테스트를 통과시키도록 add(a, b)를 가장 단순하게 구현해줘.

결과 1 passed — 이 테스트는 이제 안전 자산.

REFACTOR — 안전한 개선

▶ 프롬프트

불필요한 주석·가독성 저해 요소를 정리하고 pytest 통과를 확인해줘.

결과 여전히 1 passed — 불안하지 않다.

체험

각 단계 뒤에 !pytest -q 로 직접 결과를 확인합니다 — AI의 “통과했습니다”가 아니라 테스트 로그가 판정합니다.

혼자 도는 5단계 RED → GREEN 루프

1

요구사항 1개

너무 크면 더 잘게 쪼갠다

2

RED 테스트

Codex에게 “테스트만” 작성시킨다

3

실패 로그 확인

왜 실패하는지 눈으로 본다

4

GREEN 최소 구현

실패 로그를 Context로 줘 통과시킨다

5

커밋 분리

test 커밋과 feat 커밋을 따로 남긴다

원칙

반복의 원칙 — 한 번에 요구사항 하나. RED→GREEN→커밋을 마치기 전에 다음으로 넘어가지 않는다. 실패 로그는 매번 새로 확인한다.

ARRR — RED·GREEN·REFACTOR에 Repeat를 더한 네 박자

장바구니(6장)에 들어가기 전, 더 작은 예제로 ARRR 네 박자를 끝까지 — 여기서만 Repeat(새 요구사항으로 다음 사이클)까지 봅니다.

A = Ask (RED)

테스트 3개를 먼저 — 빈 문자열→0 · 단일 숫자 · 콤마 두 숫자. 구현이 없으니 ImportError/실패.

R = Respond (GREEN)

통과시키는 가장 단순한 구현만. 과도한 추상화·미래 대비 코드 금지.

R = Refine (REFACTOR)

동작은 그대로, 책임만 분리 — `_is_blank`(검증) / `_calculate_sum`(계산).

R = Repeat

새 요구: 세미콜론 구분자. 다시 Ask(실패 테스트)부터 → `_split_numbers` 추출.

원칙

RED→GREEN→REFACTOR 사이클을 절대 건너뛰지 않는다 · 실패를 먼저 확인한 뒤에만 구현한다 · “최소한의 코드”만 쓴다 · 리팩터링은 외부 동작을 바꾸지 않는 선에서만.

ARRR 예제 — 숫자 더하기 계산기 (Python)

빈 문자열 → 0, 단일 숫자 → 그 값, 쉼표 구분 합계. 이후 세미콜론 구분자를 Repeat로 추가한다.

A · Ask (RED)

```
def test_empty_returns_zero():
    assert add("") == 0
def test_single_number():
    assert add('5') == 5
def test_two_numbers():
    assert add('3,5') == 8
```

R · Respond (GREEN)

```
def add(numbers):
    if not numbers:
        return 0
    return sum(
        int(n) for n in numbers.split(',')
```

R · Refine (REFACTOR)

```
# 구분자 분리 로직을 함수로 추출(SRP)
def _split_numbers(numbers):
    return re.split('[,;]', numbers)
```

R · Repeat (새 RED)

```
def test_semicolon_delimiter():
    assert add('3;7') == 10 # 새 요구사항
```

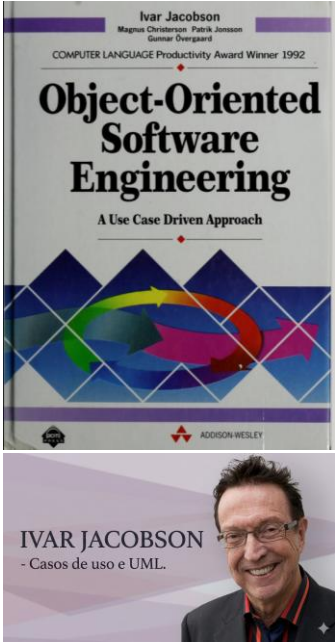
Dual-Track — 두 트랙으로 나눠 돌린다

한 화면짜리 앱이라도 “계산 규칙(Logic)”과 “화면 동작(UI)”은 성격이 다르다.

구분	Logic 트랙 (Track B)	UI 트랙 (Track A)
검증 대상	순수 함수(입력→출력) 계산 규칙	화면 렌더링·사용자 입력 흐름
이번 실습	subtotal · 문턱 할인 · VIP 계산	주문 폼 · 최종 금액 표시
속도/안정성	매우 빠르고 결정적	상대적으로 느리고 환경 영향을 받음
도구	pytest 단위 테스트	Flask test_client · 브라우저 확인 · 브라우저 MCP
ECB 계층	Entity — 외부 I/O 없음	Boundary — Entity를 재사용, 재계산하지 않음

주의

ECB는 MVC·Spring Controller와 다르다 — 분석 개념 도구이지 코딩 규칙이 아니다 (Ivar Jacobson, OOSE 1992).



Dual-Track TDD 5단계 사이클 — 두 트랙의 맞물림

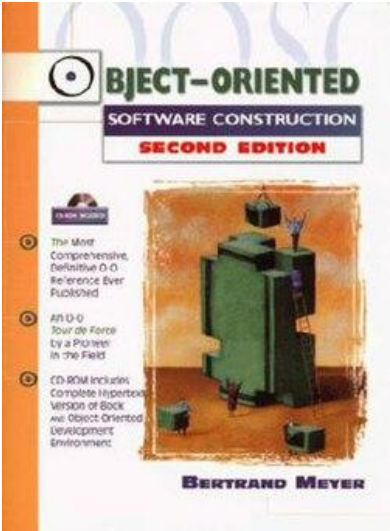
단계	Discovery 트랙	Delivery 트랙	ARRR	신호
① 가설 수립	계약·골든셋·목표 수치 정의, ID 부여	(대기)	—	계약 표
② Red	가설을 검증 가능한 조건으로 구체화	ID마다 실패 테스트(tests/만)	Ask	의도된 FAIL
③ Green	구현이 가설을 만족하는지 1차 확인	통과 최소 구현, 구현 줄에 ID 주석	Respond	전부 PASS
④ Refactor	평가 기준이 정밀해질 지점 식별	중복 제거·SSOT 추출, 계약 불변	Refine	여전히 GREEN
⑤ 합류/반복	골든셋·AARRR 지표로 전체 검증	E2E 스모크 + 다음 ID로 회귀	Repeat	매트릭스 완성

핵심

닫힌 루프 — Discovery의 산출물이 Delivery의 테스트 픽스처가 되고, Delivery의 계측이 Discovery의 다음 가설이 됩니다. 이것이 프로세스 수준의 Dual-Track입니다.

C2C의 정의와 4계층 검증 구조

C2C(Concept-to-Code) — 임의의 구현 줄에서 위로 올라가면 반드시 불변식에, 임의의 불변식에서 아래로 내려가면 테스트·구현 줄에 닿는다. 이 양방향 도달 가능성이 곧 C2C.
불변식 : 절대로 실패해서는 안 되는(high reliability)" 미션



계층	이름	질문	ECB 대응
Layer 1	Invariant (불변식)	무엇이 절대 바뀌면 안 되는가	수학적 진실
Layer 2	UI Contract Test	UX가 올바르게 반응하는가	Boundary
Layer 3	Domain Test	비즈니스 규칙이 작동하는가	Control
Layer 4	Implementation Unit	단위 알고리즘이 정확한가	Entity

주의

Layer 1(불변식)은 사슬의 정점 — 그 불변식은 어디서 오는가? 이 질문이 5장(제품 계약)으로 이어집니다. 불변식 기반 계약(Invariant-based Contract)은 계약에 의한 설계(Design by Contract, DbC) 방법론에서 유래. 베르트랑 마이어(Bertrand Meyer)의 Eiffel(1980년대 후반)과 그의 저서 《Object-Oriented Software Construction》(1988), 1997년에 출간된 개정 2판을 통해 이 개념이 최초로 정립

계약 ID(추적의 못)와 추적성 매트릭스

RED

테스트 이름·주석에 계약 ID

GREEN

구현 줄마다 계약 ID 주석

COMMIT

커밋 메시지에도 계약 ID

금지

ID 없는 동작은 구현하지 않는다

계약 ID	의미	검증 테스트	구현 위치
INV-1	subtotal = $\sum \text{price} \times \text{qty}$	test_subtotal_sums_price_times_qty	subtotal() return
INV-2	경계 포함 문턱 할인	test_threshold_applies_at_50000	apply_threshold_discount()
INV-3	문턱→VIP 순서·금액	test_vip_order_and_value	final_total()
INV-4	$0 \leq \text{final} \leq \text{subtotal}$	test_discount_never_increases	final_total() return
E-1 / E-2	None→TypeError / 음수→ValueError	test_none_raises / test_negative_raises	subtotal() 가드·루프

핵심

구현에는 있으나 매트릭스에 없는 코드 줄 = “정체불명 코드” → 제거하거나 계약화. 켄트 벡이 꼽은 AI의 첫 번째 경고 신호(과잉 구현)를 원천 차단합니다.

TDD · 리팩터링 · Tidy First — 계승하는 규율

단계	이때 만지는 것	끝나는 신호
RED	tests/ 만	테스트가 “의도대로” 실패
GREEN	src/ 만	테스트가 전부 통과
REFACTOR	src/ (필요 시 tests/)	여전히 통과 + 더 깔끔

TDD 세 원칙

- 실패를 먼저 본다
- 최소로 통과시킨다 (과잉 구현 금지)
- 통과 후 정리한다

Tidy First 3규칙

- 구조 변경과 동작 변경을 한 커밋에 섞지 않는다
- 둘 다 필요하면 구조 변경 먼저
- 전후로 테스트를 돌려 동작 불변 확인



주의

Tidy First?는 익스트림 프로그래밍(XP)과 TDD(테스트 주도 개발)의 창시자인 켄트 벡(Kent Beck)이 저술한 소프트웨어 설계 및 개발 철학에 관한 책이자 방법론.

Augmented Coding — “씨앗을 먹지 마라”

"AI에 코딩을 무심코 맡기는 것은, 농부가 배고프다고 다음 해 심을 씨앗까지 먹어 치우는 것과 같다." — 켄트 벅

	Vibe Coding	Augmented Coding
태도	돌아가면 OK, 결과를 안 본다	주도권을 쥐고 결과에 책임진다
관심사	동작만	동작 + 테스트 + 설계 품질
AI와의 관계	맡기고 기다린다	감시하며 필요하면 개입·중단

주의

AI가 결길로 새는 경고 신호 — ① 요청하지 않은 기능(과잉 구현) → “ID 없는 동작은 구현하지 않는다”가 차단 ② 테스트를 속이는 정황(비활성화·삭제·수정) → “테스트 약화 금지”가 차단. 두 신호는 4장 하네스에서 금지(Don't) 5조항으로 성문화됩니다.

프로젝트 수준 — 13 STEP · 4개 페이즈 조감도

1

기획·아이템 선정

STEP 1~5

Self-Audit · Job 검증(JTBD · Mom Test · VPC) · 아이템 선택(ICE)

2

제품 설계·지표

STEP 6~8

제품 계약(INV/E/AC + L0~L2) · AARRR 역산 · PRD 확정

3

엔지니어링·구현

STEP 9~12

대표 모듈 시연(AARRR 완주) · 하네스 구축 · 모듈별 8단계 반복

4

평가·회고

STEP 13

골든셋 평가 · AARRR 실측 · 추적성 매트릭스 점검 · KPT

핵심

모듈 수준의 8단계 골격이 프로젝트 수준에서 13 STEP으로 확장됩니다. 오늘은 STEP 6~12를 실습하고, 나머지는 8장 Magic Square와 프로젝트 가이드(D2)로 이어집니다.

02



CHAPTER 2

단위 테스트 설계

좋은 테스트는 무엇을, 어떤 구조로 검증하는가 — F.I.R.S.T · Right-BICEP · AAA · Test Smell · 테스트 케이스와 커버리지.

🕒 1교시 · 이론

단위 테스트 원칙 — F.I.R.S.T (Python)

테스트가 없는 코드를 수정하면?

내가 고친 코드가 어떤 영향을 끼칠지 몰라 수정이 어렵다. 결국 코드는 썩는다.

테스트가 잘 갖춰진 코드를 수정하면?

잘못 고쳐도 테스트가 잡아 주니 편하게 수정한다. 리팩토링 자유도가 높아진다.

원칙	설명	Python (pytest)
Fast	테스트는 빨리 돌아야 한다	함수 단위 — ms 단위 실행
Independent	테스트는 서로 의존하면 안 된다	fixture로 초기화 (@pytest.fixture)
Repeatable	어떤 환경에서도 같은 결과	@pytest.mark.parametrize · 고정 시드
Self-Validating	결과는 bool로 자동 판단	assert expected == actual
Timely	테스트는 실 코드 직전에 작성	TDD — 테스트 함수 먼저 작성

Right-BICEP & CORRECT — 무엇을 테스트할 것인가 (Python)

Right-BICEP — 결과 검증 6관점

- R** Right 결과가 옳은가? `assert add(5,5) == 10`
- B** Boundary 경계조건? `sys.maxsize, 0, None`
- I** Inverse 역관계? `sorted` → 역순 후 원본
- C** Cross-check 다른 수단으로 교차 검증?
- E** Error 에러 조건? `pytest.raises`
- P** Performance 성능 조건? 타임아웃 측정

CORRECT — 경계 조건 7관점

- C** Conformance 형식 준수: 이메일·날짜·JSON
- O** Ordering 정렬·순서: 오름차순 보장
- R** Range 범위: 나이 0~150, 인덱스
- R** Reference 참조 유효성: 없는 ID·삭제된 객체
- E** Existence 존재: None·빈 문자열·0
- C** Cardinality 개수: 최소 1, 최대 100
- T** Time 시간: 만료·타임아웃·동기화

AAA 패턴 — 좋은 테스트의 구조 (Python)

모든 테스트는 Arrange(준비) → Act(실행) → Assert(검증) 3단계로 명확히 구분한다.

Python — pytest

```
def test_mean_score_is_calculated():
    # Arrange — 준비
    collection = ScoreCollection()
    collection.add(5)
    collection.add(7)

    # Act — 실행
    result = collection.arithmetic_mean()

    # Assert — 검증
    assert result == 6
```

AAA 단계	pytest	핵심 원칙
Arrange	fixture / 객체 생성	테스트 픽스처 분리
Act	메서드 호출 1줄	단 하나의 동작
Assert	assert 문	명확한 성공/실패

3분할이 무너질 때

Act가 두 줄 이상이면 한 테스트가 두 동작을 검증하고 있다는 신호 — 테스트를 쪼갭니다(→ 2.4 Eager Test).

Write Better Tests — 5 Steps

1 Production 코드처럼

테스트 코드도 실제 코드처럼 다룬다 — 읽기 쉽고 유지보수 가능하게.

2 패턴으로 가독성

AAA 등 테스트 패턴으로 의도를 명확히 드러낸다.

3 불안정 테스트 피하기

시간·랜덤·외부 의존을 제거해 Flaky를 없앤다.

4 적절한 수준 선택

단위·통합·E2E 중 검증 목적에 맞는 수준을 고른다.

5 Test Double 활용

핵심 외부 의존만 Fake/Mock으로 격리, 나머지는 실제로.

+ Test Smell 경계

다섯 원칙이 무너지는 지점이 곧 다음 슬라이드의 6대 냄새.

Test Smell — 6가지 대표 유형

Mystery Guest

- ✗ 외부 파일·DB 직접 의존 → 실패 원인 불명확
- ✓ 데이터 인라인 / Fake 객체로 격리

Flaky Test

- ✗ 시간·랜덤·전역 상태 의존 → 실행마다 결과 다름
- ✓ monkeypatch·freezegun으로 고정, fixture 독립

Assertion Roulette

- ✗ assert 여러 개 → 실패 위치 불명확
- ✓ assert 메시지 추가 · 검증 대상별로 분리

Eager Test

- ✗ 한 테스트에 기능 여럿 → SRP 위반
- ✓ 한 테스트 = 한 관심사로 분리

Setup Bloat

- ✗ fixture가 비대 → 의도 파악 어려움
- ✓ 각 테스트에 필요한 준비만

Over-mocking

- ✗ 모든 것을 Mock → 실제 동작 미검증
- ✓ 핵심 외부 의존만 Mock, 나머지는 실제

원칙 테스트 코드도 Production 코드처럼: 읽기 쉽고, 빠르고, 독립적이어야 한다. AAA 3분할이 무너지는 곳에서 이 냄새들이 자랍니다.

테스트 케이스란 무엇인가

테스트 케이스(Test Case)란 요구사항을 준수하는지 검증하기 위하여 테스트 조건, 입력값, 예상 출력값 및 수행 결과 등을 명세한 것이다.

테스트 케이스 = 특정 조건 + 입력값 + 기대 결과

1 실행 사전 조건

테스트를 수행하기 위한 전제 상태

2 입력값의 묶음

특정 프로그램 경로를 실행하도록 설계된 입력 집합

3 기대 결과·사후 조건

요구사항과 일치하는지 판정할 예상 출력

테스트 케이스 작성 절차 — 사칙연산 예 (Python)

테스트 단계	입력값	예상값	중요도	결과
1 + 10	1, 10	11	상	성공
0 + 1	0, 1	1	상	성공
0 ÷ 0	0, 0	ZeroDivisionError 발생	상	성공
-1 + (-10)	-1, -10	-11	중	성공

테스트 환경

- Python 3.11+
- pytest 8.x
- VS Code + 통합 터미널
- Windows / macOS / Linux

```
import pytest
from src.utils.math_utils import add, divide

def test_add_positive():
    assert add(1, 10) == 11

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(0, 0)
```

테스트 케이스 작성 절차 — 목록 양식

ID	구분	수행 절차	기대 결과	추적성	중요도
1.1	우편번호 조회	5자 이상 입력 후 조회 클릭 → 조회 아이콘 클릭	팝업 표시 후 데이터 출력 / 없으면 안내 메시지	요구사항 ID-100	중
...	...	...	...	...	...

좋은 테스트 케이스의 요건

사전조건 · 수행절차 · 기대결과 · 결과 · 추적성 · 중요도 · 비고를 갖춘 검사 가능한 산출물이어야 한다 — 특히 “추적성” 칸이 요구사항 ID와 케이스를 이어 준다.

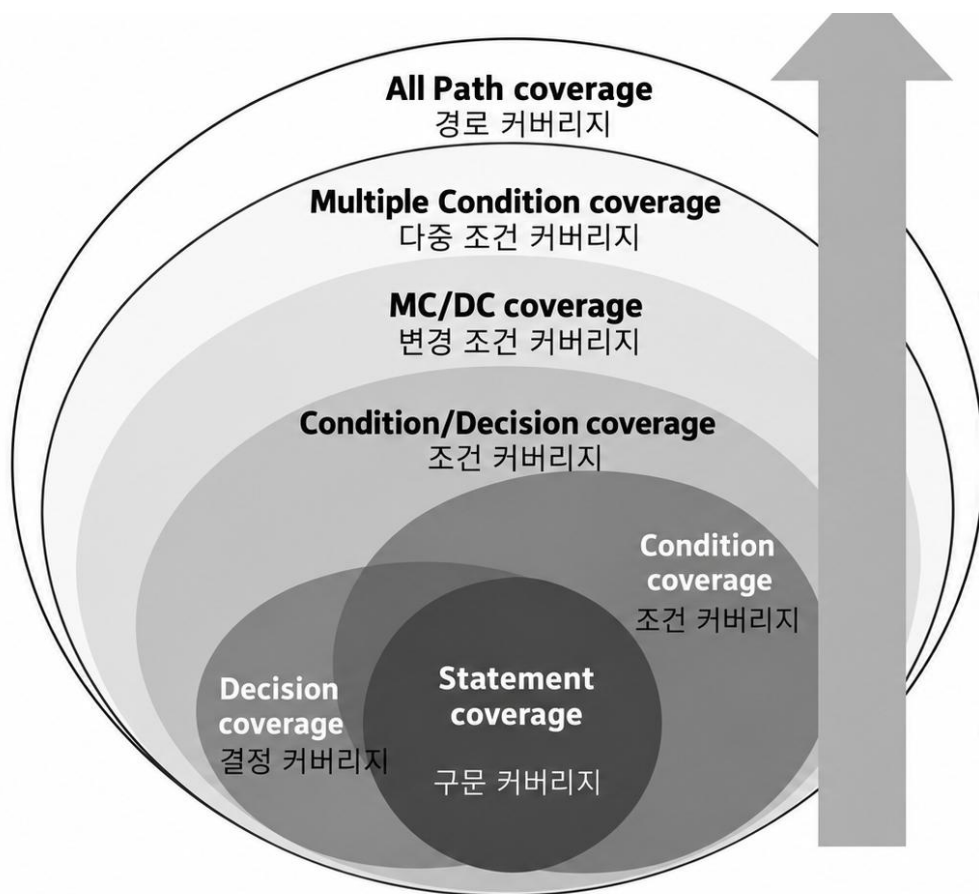
핵심

이 “추적성” 열이 6장의 추적성 매트릭스와 동일한 역할을 합니다 — 어떤 요구사항이 어떤 케이스로 검증되는지 한눈에 드러나야 합니다.

테스트 방법론 — 테스트 목적에 따른 분류

방법론	설명	사례
동적 테스트	프로그램 실행을 요구하는 테스트	화이트박스, 블랙박스
정적 테스트	프로그램 실행 없이 구조를 분석하여 논리성 검증	코드 검사, 워크스루
화이트박스 테스트	프로그램 내부 로직을 보면서 테스트 (구조 테스트)	구조 테스트, 루프 테스트
블랙박스 테스트	프로그램 외부 명세를 보면서 테스트 (기능 테스트)	동등분할, 경계값 분석, 원인-결과 그래프, 오류 예측
하드웨어 테스트	하드웨어의 성능과 신뢰성 테스트	부하 테스트, 스트레스 테스트
소프트웨어 테스트	이해관계자에게 시험 대상 제품·서비스의 품질 정보를 제공하는 조사 과정	사람은 코드의 모든 결함을 찾을 수 없다 — 전략이 필요

Test Coverage — 커버리지 수준과 비용



- 커버리지 100% 달성에 필요한 테스트 케이스 수 증가
- 코드 기준 테스트 범위가 증가 → 식별 결함 수 증가
- 테스트 실행 비용 증가

읽는 법

- 안쪽(구문)에서 바깥(전체 경로)으로 갈수록 기준이 엄격해진다
- 커버리지 100%에 필요한 테스트 케이스 수가 급증한다
- 코드 기준 테스트 범위가 넓어질수록 발견하는 결함은 늘지만, 실행 비용도 함께 증가한다
- `pytest --cov=src`로 측정 — 목표 수치는 팀이 합의

테스트 목표 설정 — OKR · SMART · 측정 지표

SMART	설명
Specific	구체적
Measurable	측정 가능
Attainable	달성 가능
Realistic	현실적
Time-based	시간 제한

OKR — 목표는 영감, 핵심 결과는 실행

넓은 비전(Objective)을 세우되, 이를 달성할 구체적·측정 가능한 지표(Key Results)를 붙인다. 예: “Gmail 성공적 런칭” → 베타 100만 명 · 전송 0.1초 이하 · 만족도 90점.

측정 지표	수식
테스트 진척율	통과된 케이스 ÷ 생성된 케이스
결함 조치율	수정 완료 결함 ÷ 발견된 결함
요구사항 커버리지	케이스로 확인된 요구사항 ÷ 명세된 요구사항

핵심

품질 목표는 SMART로 세우고 측정 지표로 검증한다 — “요구사항 커버리지”가 100%에 달하는지 곧 추적성 매트릭스에 빈칸이 없는지와 같은 질문입니다.

Unit Test — 경계값 하나를 RED → GREEN으로

RED — 의도된 실패

▶ 프롬프트

tests/test_calc.py 에 divide(1, 0) 이 ZeroDivisionError 를 던지는지 검증하는 테스트만 추가해줘. src/ 는 수정하지 마.

결과 divide가 없거나 예외를 안 던지면 pytest는 의도된 실패 (RED).

체험

Right-BICEP의 E(Error)와 CORRECT의 Existence를 한 번에 체험합니다 — 테스트 이름에 무엇을 검증하는지 드러나게 짓습니다 (test_divide_by_zero_raises).

GREEN — 최소 통과

▶ 프롬프트

!pytest -q 실패 로그를 보고, 방금 테스트만 통과시키는 최소 구현을 해줘. 다른 기능은 추가하지 마.

결과 2 passed — 경계 조건도 이제 안전자산.

03



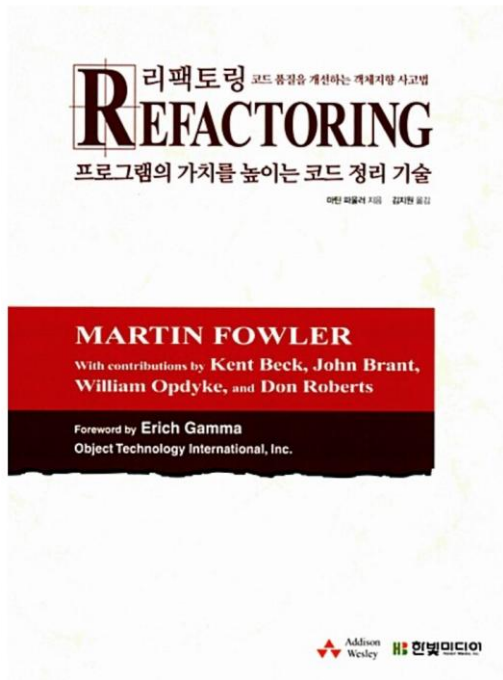
CHAPTER 3

클린 코드와 리팩토링

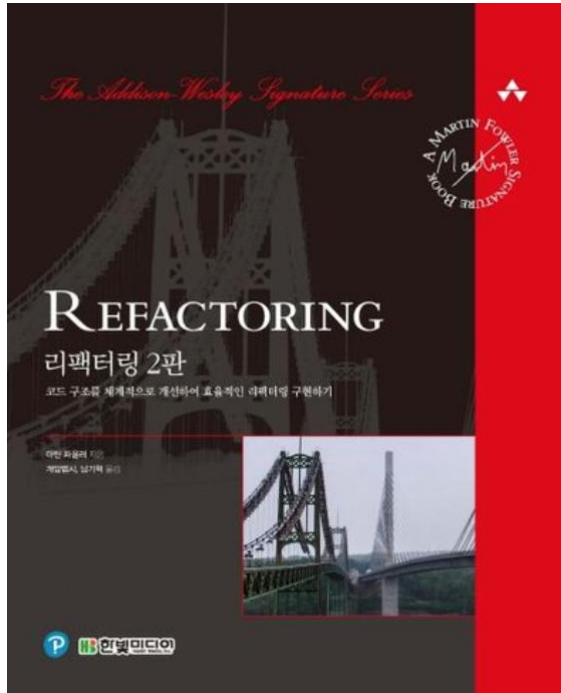
24가지 코드 냄새 · 리팩토링 기법 · SOLID · 네이밍 · 함수 설계 · 오류 처리 · 정적 분석 (Python). 외부 동작을 바꾸지 않으면서 내부 구조를 개선한다.

🕒 2교시 · 이론

Code Smell & Refactoring — 무엇을, 왜 고치는가



Martin Fowler, 『리팩토링』 1판·2판



리팩토링이란

외부 동작을 바꾸지 않으면서 내부 구조를 개선하는 것. 테스트가 GREEN인 상태에서만 수행한다.

코드 냄새(Code Smell)란

당장 버그는 아니지만 더 깊은 설계 문제를 암시하는 징후. 냄새를 찾아 이름 붙이면 처방이 따라온다.

이 장의 흐름

냄새 24가지 → 3의 법칙 → SOLID → 네이밍·함수·오류 처리 → 정적 분석 → QA 프롬프트

24가지 코드 냄새 (Fowler)

1 이해하기 힘든 이름

2 중복 코드

3 긴 함수

4 긴 매개변수 목록

5 전역 데이터

6 가변 데이터

7 뒤엎킨 변경

8 산탄총 수술

9 기능 편애

10 데이터 뭉치

11 기본형 집착

12 반복되는 switch

13 반복문

14 성의 없는 요소

15 추측성 일반화

16 임시 필드

17 메시지 체인

18 중재자

19 내부자 거래

20 거대한 클래스

21 서로 다른 인터페이스의 대안 클래스들

22 데이터 클래스

23 상속 포기

24 주석

코드 스멜 주요 유형과 처방

1 중복 코드

같은 코드 두 곳 이상
→ 함수 추출 / 공통
유틸

2 긴 함수

20줄 초과 → 기능별
분리

3 거대한 클래스

너무 많은 책임 →
SRP 적용, 분리

4 긴 매개변수 목록

3개 초과 → 매개변수
객체 도입

5 죽은 코드

미사용 코드 → 즉시
삭제 (Git이 기억)

6 매직 넘버

의미 불명 값 → 이름
있는 상수로 추출

7 깊은 중첩

3단계 이상 → Early
Return / 함수 추출

8 부적절한 이름

a, tmp, data → 의도
를 드러내는 이름

3의 법칙 (Rule of Three) — Python 예제

돈 로버츠(Don Roberts): ① 처음엔 그냥 한다 ② 두 번째 중복은 일단 진행한다 ③ 세 번째 반복되면 리팩토링한다.

참고 : <https://eoinnoble.com/posts/origins-of-the-rule-of-three/>



X 세 번째 중복 (냄새)

```
class UserService:
    def get_user_full_name(self, first, last):
        return f'{first} {last}'

    def get_admin_full_name(self, first, last):
        return f'{first} {last}' # 중복

    def get_guest_full_name(self, first, last):
        return f'{first} {last}' # 세 번째!
```

✓ 공통 로직 추출

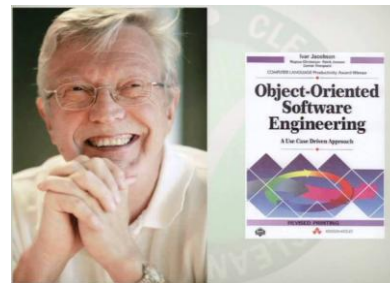
```
def format_full_name(first, last):
    return f'{first} {last}'

class UserService:
    def get_user_full_name(self, f, l):
        return format_full_name(f, l)

    def get_admin_full_name(self, f, l):
        return format_full_name(f, l)
    # 이름 관련 로직을 한 곳으로 위임
```

객체 지향 설계 5원칙 — SOLID

로버트 마틴(Robert C. Martin, 별칭 Uncle Bob) 2002년 저서 《Agile Software Development, Principles, Patterns, and Practices》에서 정립. 2004년경 마이클 페더스(Michael Feathers)가 SOLID라는 두문자어(Acronym)로 명명.



S

단일 책임 (SRP)

하나의 클래스는 하나의 책임만 가진다

2002년 저서 《Agile Software Development, Principles, Patterns, and Practices》에서 정립

O

개방-폐쇄 (OCP)

확장에는 열려 있고 변경에는 닫혀 있다

1988년 바트랑 마이어(Bertrand Meyer)가 처음 제안한 개념으로, 이후 로버트 마틴이 객체 지향 관점으로 재정립

L

리스코프 치환 (LSP)

파생 클래스는 기반 클래스를 대체할 수 있어야 한다

1987년 바바라 리스코프(Barbara Liskov)가 발표한 강연/논문 내용을 1996년 로버트 마틴이 원칙으로 정리

I

인터페이스 분리 (ISP)

쓰지 않는 메서드에 의존하지 않는다

1996년 로버트 마틴이 제록스(Xerox) 컨설팅 당시 체계화한 원칙

D

의존 역전 (DIP)

추상화에 의존하고 구체적인 것에 의존하지 않는다

1996년 로버트 마틴이 발표한 논문 《The Dependency Inversion Principle》에서 처음 발표

SRP 예시 — 책임을 분리한다 (Python)

클래스는 단 하나의 책임만 가진다 — 변경의 이유가 하나여야 한다. 직렬화·표현 책임은 별도로 분리한다.

X SRP 위반

```
class Resume:
    def introduce_me(self): ...
    def get_grade(self): ...
    def to_json(self): ...    # 직렬화?
    def to_xml(self): ...    # 직렬화?
    def to_html(self): ...   # 표현/UI?
```

✓ SRP 준수

```
class Resume:
    def introduce_me(self): ...
    def get_grade(self): ...

class Converter(Protocol):
    def convert(self, r: Resume) -> str: ...

class JsonConverter:    # Converter 구현
    def convert(self, r): ...
```

OCP · LSP · ISP · DIP — Python 적용 요약

원칙	위반 신호	Python 해법
OCP	새 유형마다 if/elif 추가	추상 기반 클래스(ABC)·Protocol + 딕셔너리 디스패치
LSP	자식이 부모 계약을 깬 (fly 못하는 Ostrich)	공통은 Bird, 능력은 Flyable 인터페이스로 분리
ISP	쓰지 않는 메서드 강제 구현 (Penguin.fly)	Bird / FlyableBird로 인터페이스 분리
DIP	구체 클래스에 직접 의존 (EmailService)	MessageService 추상화 + 생성자 주입(DI)

핵심

Python에서는 `abc.ABC`·`typing.Protocol`로 추상화를 표현하고, 생성자 주입으로 의존을 역전한다. 다형성과 덕 타이핑이 SOLID 적용을 자연스럽게 해 준다.

좋은 네이밍의 원칙 (Python)

의도를 표현하라

✗ `d` # 경과 일수

✓ `elapsed_days`

그릇된 정보 피하기

✗ `account_list` # List 아닐 수도

✓ `accounts`

의미 있는 구분

✗ `a1, a2 / product_info`

✓ `source, destination`

검색 가능한 이름

✗ `if s == 5:`

✓ `WORK_DAYS = 5`

클래스 = 명사

✗ `class Manager / Processor`

✓ `class Customer / Account`

함수 = 동사

✗ `name() / age()`

✓ `get_name() / is_valid()`

Python 네이밍 & 스타일 — PEP 8

구분	규칙 (PEP 8)	예시
클래스	PascalCase	CalculatorService
함수 / 메서드	snake_case	calculate_total()
변수	snake_case	elapsed_time
상수	UPPER_SNAKE_CASE	MAX_SIZE
비공개 멤버	선행 밑줄	_internal_state
모듈 / 패키지	짧은 소문자	cart, utils.math_utils
들여쓰기	스페이스 4칸	탭 대신 스페이스
최대 라인 길이	79자 (docstring 72자)	black / ruff 로 자동 정렬

핵심 VS Code: ruff / black / flake8 로 스타일을 자동 강제할 수 있다 — 사람이 아니라 도구가 스타일을 지킵니다.

함수 설계 — 작게, 하나만, 한 수준으로

1 작게 만들어라

20줄 이내가 이상적. 들여쓰기
1~2단계 이내

2 한 가지만 해라

함수 이름으로 설명되는 한 가
지 작업만

3 추상화 수준 통일

고수준(비즈니스)과 저수준
(I/O)을 섞지 말 것

4 서술적 이름

길고 서술적인 이름이 짧고 의
미 없는 이름보다 낫다

5 인수를 최소화

이상 0개, 최대 3개. 초과 시 매
개 변수 객체 고려

6 부수 효과 없애기

숨겨진 변경 금지. 명령과 조회
를 분리(CQS)

데미터 법칙 — 최소 지식 원칙 (Python)

Law of Demeter :

- 낯선 이에게 말 걸지 않는다 (Don't talk to strangers)-1987년 말 미국 노스이스턴 대학교(Northeastern University)의 이안 홀랜드(Ian Holland)와 동료들이 진행하던 '데미터 프로젝트(Demeter Project)' 에서 사용한 공식 **모토(Motto)**
- 연쇄 호출은 기차(Train wreck)이자 결합도의 냄새(Smell)다 - 마틴 파울러(Martin Fowler)의 저서 《리팩터링(Refactoring)》과 로버트 C. 마틴(Uncle Bob)의 저서 《클린 코드(Clean Code)》를 통해 소개

X 나쁜 예 (위반)

```
city = order.customer \
        .address \
        .city
```

- order→customer→address→city 내부 구조를 다 안다
- 구조가 바뀌면 호출부가 전부 깨진다

✓ 좋은 예 (준수)

```
city = order.customer_city()
# Order 내부에서 위임 처리:
# return self._customer.address.city
```

- 외부는 Order 하나만 알면 된다
- 내부 구조 변경이 호출부에 새어 나가지 않는다

원칙

위임 메서드로 감싸 내부 구조를 숨긴다 — 객체는 자신의 직접 이웃하고만 대화한다.

Clean Error Handling (Python)

원칙	나쁜 코드 ✕	좋은 코드 ✓
오류 코드 대신 예외	if delete_page(p) == E_OK:	try: delete_page(p) / except ...:
None 반환하지 않기	return None # 책임 떠넘김	빈 리스트 · 명시적 예외
None 전달하지 않기	do_something(None)	가드 절 · Optional 명시
예외에 의미 담기	raise Exception("error")	raise StorageError(...) from e

```
# 예외 체이닝으로 원인 보존 (raise ... from e)
def read_config(path):
    try:
        return _parse(path)
    except OSError as e:
        raise StorageError(f'설정 읽기 실패: {path}') from e
```

정적 분석으로 코드 스멜을 자동 탐지 (Python)

정적 분석은 실행 없이 소스코드를 분석해 버그·스타일 위반·복잡도 문제를 찾아낸다.

도구	탐지 내용	연동 방법
Pylance	실시간 타입·문법 경고	VS Code Python 확장 내장
ruff	스타일·버그·복잡도 (초고속)	ruff check . / pre-commit
flake8	코딩 규칙 (PEP 8·복잡도)	flake8 + 설정 파일
mypy	정적 타입 검사	mypy src
bandit	보안 취약 패턴	bandit -r src
pytest-cov	테스트 커버리지 측정	pytest --cov=src

```
$ ruff check . && mypy src && pytest --cov=src # 한 파이프로 정적 분석 자동 실행
```

QA를 위한 프롬프트 엔지니어링 — 5요소

막연한 “고쳐줘”는 범위가 폭주한다. QA 프롬프트는 아래 5요소를 명시할수록 결과가 안정적이다.

1

역할/관점

“보안 리뷰어 관점에서”

2

맥락

“calc 함수만, 동작 변경 없이”

3

작업

“스멜을 심각도 순으로 나열”

4

제약

“파일 수정 말고 보고만”

5

출력 형식

“표: 위치 | 문제 | 위험 | 제안”

핵심

출력 형식을 지정하면 결과를 그대로 QA 리포트·커스텀 스킴 절차로 재활용할 수 있다 — 형식은 자동화의 입력이 된다.

작업별 프롬프트 패턴 — 진단 · 리뷰 · 리팩터 · 테스트

▶ 진단

«대상»의 코드 스멜·중복·복잡도를 찾아줘. 수정 말고 [위치|유형|근거|난이도] 표로, 우선순위 순으로 정리해줘.

▶ 리뷰

방금 변경을 «보안|회귀|성능» 관점으로만 리뷰해줘. 동작이 바뀔 지점과 그걸 잡을 테스트가 없는 부분을 우선순위로.

▶ 리팩터

«함수»의 외부 동작을 바꾸지 말고 내부 구조만 개선해줘. 먼저 계획을 보여주고 승인 후 적용, 매 단계 후 테스트 실행.

▶ 테스트

«함수»의 테스트 갭을 찾아줘(경계·예외·중복). 빠진 케이스를 먼저 보여주고, 합의된 케이스만 테스트로 추가해줘.

원칙

범위 폭주를 막는 한 줄 — “먼저 계획을 보여주고 승인 후 진행. 요청하지 않은 변경은 하지 마.”

Code Smell & Refactoring — 진단은 읽기 전용, 수정은 한 번에 하나

① 읽기 전용으로 진단

```
$ codex --sandbox read-only
# 읽기 전용 세션: 파일 수정 불가
```

▶ 진단 (read-only 모드)

@src/user_service.py 의 코드 스멜을 찾아줘. 수정하지 말고 [위치 | 스멜 | 근거 | 처방] 표로, 우선순위 순으로 보고해줘.

▶ 리팩터 (기본 Auto 모드)

1순위 스멜(중복 코드 — 3의 법칙) 하나만 고쳐줘.
format_full_name() 으로 추출하고 외부 동작은 바꾸지 마. 적용 전 계획을 먼저 보여주고, 끝나면 !pytest -q 결과를 보고해.

확인할 것

- pytest가 리팩터 전후 모두 GREEN인가
- 테스트 파일이 수정되지 않았는가
- 한 번에 스멜 하나만 처리했는가 (Tidy First)

체험

탐지(read-only)와 실행(Auto)을 분리하면 “고치다 보니 다른 것도 바꿨다”는 범위 폭주가 사라집니다 — 8장 리팩터링 실습(8.8)의 기본 리듬입니다.



PART II

실습 — 장바구니 할인 계산기

“장바구니 할인 계산기”를 TDD로 완성합니다. Logic 트랙(subtotal → 문턱 할인 → VIP · pytest) + UI 트랙(주문 폼 → 최종 금액 · Flask).

04



CHAPTER 4

실습 준비와 하네스 엔지니어링

Agent = Model + Harness. 프로젝트를 초기화하고, 프롬프트 c 컨텍스트 c 하네스의 3수준 사다리를 세워 AI가
결길로 새지 못하게 구조로 막습니다.

🕒 3교시 · 실습 준비 · 하네스

무엇을 만드나 — 장바구니 할인 계산기, 두 트랙으로

Track B — Logic (Entity)

금액·할인 계산(순수 함수) — 합계 → 문턱 할인 → VIP 할인

INV-1 · INV-2 · INV-3 · INV-4 · E-1 · E-2

tests/entity/

상류(5장) — Mom Test 메모에서 제품 계약(INV/E/AC + 증거 수준)을 도출

Track A — UI (Boundary)

주문 폼 웹페이지(Flask) — 수량·VIP 입력 → 최종 금액 표시

UC-1 · UC-2 · UE-1

tests/boundary/

하류 — AARRR 계측 인터페이스(M-1 · M-2)는 D2 프로젝트에서 계약 ID별 AARRR로 확장

주의

ECB 방향 단속 — Boundary는 Entity를 import 할 수 있지만, Entity는 Flask를 절대 import 하지 않습니다. 이 방향이 깨지면 Layer 4 검증이 외부 의존에 오염됩니다.

실습의 뼈대 — Discovery → Delivery 8단계 골격

Discovery — 코드 작성 전에 끝낸다

- ① **목표**
보장할 것 + ECB 역할·C2C 계층·트랙 좌표를 못 박는다
- ② **PRD(실패 조건)**
기능 목록이 아니라 “무엇이 실패인가”를 L0~L3로
- ③ **계약 식별+ID**
INV-*·E-*에 ID 부여 (추적의 못)
- ④ **스켈레톤**
시그니처 고정 + TODO(계약 ID). 완성 코드는 감춘다

Delivery — ⑤~⑧이 ARRR 사이클

- ⑤ **A = Ask (RED)**
ID마다 실패 테스트를 tests/에만. src/ 금지
- ⑥ **R = Respond (GREEN)**
실패 로그 → 최소 구현. 구현 줄에 ID 주석
- ⑦ **R = Refine (REFACTOR)**
중복 제거·SSOT. 동작·계약 불변
- ⑧ **R = Repeat**
다음 계약 ID로 다음 RED + 회귀(pytest -q)

원칙

절대 곧바로 코딩하지 않습니다. ④ 스켈레톤에서 완성 코드를 감추는 것은 교육 장치이자 추적성 장치 — 학습자와 AI 모두 “계약→테스트→구현 줄” 순서로만 코드에 도달합니다.

파이썬 프로젝트 초기화 — 가상환경과 패키지

터미널

```
# 프로젝트 폴더 만들기
$ mkdir codex-c2c-cart && cd codex-c2c-cart
$ code . # VS Code로 이 폴더 열기

# 파이썬 가상환경 생성/활성화
$ python -m venv .venv
$ source .venv/Scripts/activate # Windows Git Bash
# macOS/Linux: source .venv/bin/activate

# 실습 패키지 설치 및 고정
$ pip install flask pytest
$ pip freeze > requirements.txt
```

✓ 성공 신호

- 프롬프트 앞에 (.venv)가 보인다
- requirements.txt가 생성됐다

자주 하는 실수

가상환경을 켜지 않고 pip install → pytest가 전역에 깔려 “왜 import가 안 되지?”로 이어진다.

핵심

실습 세션을 새로 열 때마다 가상환경 활성화가 첫 명령입니다 — 6장·7장 시작 전 체크포인트에 매번 등장합니다.

GitHub 저장소 연결 — 처음부터 Git을 켜다

터미널

```
$ git init
# GitHub 웹에서 빈 저장소 생성 후:
$ git remote add origin <저장소 주소>
$ git add . && git commit -m "chore: 초기 스캐폴딩"
$ git push -u origin main
```

앞으로 남길 커밋의 형식

```
test(entity): INV-1 ... [RED]
feat(entity): INV-1 ... [GREEN]
refactor(entity): SSOT [REFACTOR]
docs: 제품 계약 SSOT (INV-1~4)
```

형식 <타입>(<계층>): <계약 ID> <무엇> [단계]

핵심

커밋 메시지에 계약 ID를 적는 규율(4.10절)이 “버전 이력 = 추적성 매트릭스의 시간축 사본”을 만듭니다. git log --oneline만으로 어느 계약을, 어느 단계에서 다뤘는지가 추적됩니다(9.1절에서 재확인).

실습 환경 최종 체크리스트 — 모두 OK면 하네스로

- ☐ 터미널에서 codex가 실행되고 ChatGPT 계정으로 로그인되어 있다
- ☐ VS Code에서 Codex IDE 확장의 Chat·Agent 모드가 열린다
- ☐ node --version이 20 이상, python --version이 3.11 이상을 출력한다
- ☐ pip install flask pytest가 오류 없이 끝났고, 프롬프트 앞에 (.venv)가 보인다
- ☐ git status가 정상 동작하고, 원격 저장소가 연결돼 있다

핵심

하나라도 체크되지 않으면 지금 해결하고 넘어가세요. 실습 중 환경 문제로 막히는 시간이 가장 아깝습니다 — 이제 이 폴더 위에 하네스를 세웁니다.

3수준 사다리 — 프롬프트 ⊂ 컨텍스트 ⊂ 하네스

① 프롬프트 엔지니어링

지시문 자체 — 역할·작업·출력 형식(개별 요청의 품질).

실체

세션에 입력하는 한 번의 요청 · 프롬프트 박스

② 컨텍스트 엔지니어링

모델이 보는 정보 환경 — “지금 상황”을 정확히 주기.

실체

@파일 참조 · AGENTS.md 계층 병합 · 테스트 실패 로그

③ 하네스 엔지니어링

실행 골격 전체 — 세션을 초월해 상시 작동하는 구조물.

실체

pytest 하네스 · AGENTS.md · config.toml · 스킬 · MCP · 금지 규칙 (rules)

원칙

Agent = Model + Harness — LLM은 엔진일 뿐. 컨텍스트의 왕 = 테스트 실패 로그. 실패 로그 + 최소 구현 요구 = Respond의 표준형이며, 이것이 과잉 구현을 억제합니다.

프롬프트 5트랙 — 입력 채널 매핑

트랙	언제 쓰는가	입력 채널	프롬프트 패턴(요지)
Discovery	계약·정책·지표 결정	codex 세션 / IDE 확장 Chat	매트릭스로 제안하고 각 칸의 근거를 한 줄씩
Red (Ask)	새 계약 ID 착수(항상 먼저)	커스텀 스킴 \$tdd-red	INV-*를 검증하는 테스트만. tests/만 수정
Green (Respond)	Red 테스트 통과	codex 세션 / IDE Agent 모드	[실패 로그] 최소 구현만. 구현 줄에 ID 주석
Refactor (Refine)	전부 GREEN 이후 정리	/plan 모드 리뷰 후 실행	동작·계약 불변으로 SSOT 추출. 깨지면 멈춰
운영/그로스	지표 설계·퍼널 진단	codex 세션	최대 이탈 단계, 원인 가설, 다음 실험 한 가지

핵심

모든 프롬프트는 이 다섯 트랙 중 하나입니다. 지금 내가 어느 트랙에 있는지 모른 채 던지는 요청이 바로 바이브 코딩입니다. (IDE 확장: Chat = 설계 대화 · Agent = 파일 편집(RED/GREEN) · Full Access = 격리 저장소에서만)

하네스의 두 축 — ECB로 나눈 폴더 + 작업 규칙의 “지도”

프로젝트 구조 (ECB 계층별)

AGENTS.md

작업 규칙·컨텍스트 (TDD 정책의 지도)

pytest.ini · conftest.py

하네스 설정 · import 경로

src/cart.py

Entity — 금액·할인 순수 함수 (Track B)

src/app.py

Boundary — Flask 주문 폼 (Track A)

tests/entity/

Track B 계약 검증 (INV-*, E-*)

tests/boundary/

Track A 계약 검증 (UC-*, UE-*)

AGENTS.md 작성 5원칙

저장소 루트에 두면 Codex CLI·IDE 확장이 세션 시작 시 자동으로 읽는 지침 파일. 백과사전이 아니라 “짧은 지도”로 씁니다.

1 짧게 — 30줄이 300줄보다 낫다

2 테스트 “명령”을 명시한다

3 “성격”이 아니라 “행동”을 적는다

4 금지(Don't) 목록을 둔다

5 자세한 건 docs/로, 여기엔 “어디를 보라”만

프로젝트 구조 만들기 — 폴더가 곧 계층 선언

목표 구조

```
codex-c2c-cart/
├── AGENTS.md          # 하네스의 "지도"
├── .codex/
│   ├── config.toml    # 승인 정책·MCP·혹
│   └── rules/         # 명령 규칙(.rules)
├── .agents/skills/    # 커스텀 스킬(SKILL.md)
├── docs/PRD.md        # 제품 계약 SSOT — 5장
├── pytest.ini         # 테스트 하네스(트랙 마커)
├── conftest.py        # src 임포트 경로
├── src/  cart.py(Entity) · app.py(Boundary)
└── tests/ entity/ · boundary/ · golden/
```

한 번에 만들기

```
$ mkdir -p src tests/entity \
  tests/boundary tests/golden \
  .codex/rules .agents/skills docs
$ touch src/__init__.py src/cart.py
```

테스트를 ECB 계층별로 나눠 두면 “어느 계층의 계약을 검증하는지”가 폴더만 봐도 드러납니다.
cart.py는 비워 둡니다 — RED에서 실패를 보기 위해.

핵심

src/cart.py를 일부러 비워 두는 것이 6장 RED의 출발점입니다 — 실패를 눈으로 보지 않은 테스트는 검증하는 테스트인지 알 수 없습니다.

Test Harness 구성(pytest) — 트랙 마커 분리

pytest.ini

```
[pytest]
testpaths = tests
python_files = test_*.py
markers =
    entity: Track B — 불변식(INV-*)
    boundary: Track A — 입력/UI(E-*, UC-*)
    slow: 외부 호출 통합/E2E
    golden: 회귀 기준선(동작 동일성)
```

confest.py

```
# 프로젝트 루트를 import 경로에 추가
import os, sys
sys.path.insert(0, os.path.dirname(__file__))

# ModuleNotFoundError: 'src'
# → 이 파일이 루트에 없어 생기는 1순위 오류
```

확인

pytest 실행 시 “no tests ran” = 정상(에러 아님) · tests/entity/와 tests/boundary/ 폴더 존재. 외부 의존은 slow 마커로 분리 — 이 분리가 Agent의 자동 반복(Repeat) 루프의 전제입니다.

AGENTS.md — 작업 규칙을 “지도”로

AGENTS.md

```
# Repository instructions
## 구조(ECB)
- cart.py: Entity. Flask import 금지
- app.py: Boundary. cart.py 재사용
## 계약(C2C)
- SSOT는 docs/PRD.md
- ID 없는 동작은 구현하지 않는다
## 워크플로 — ARRR
- "실패하는 테스트"가 항상 구현보다 앞선다
- 테스트 실행: pytest -q
```

금지 5조항(Don't) — 하네스의 핵심

- ① 테스트 수정·삭제·skip으로 우회 금지
- ② 요청하지 않은 기능 금지(과잉 구현)
- ③ 함수 시그니처 임의 변경 금지
- ④ ID 없는 동작 구현 금지
- ⑤ 골든 마스터 파일 임의 재생성·수정 금지

핵심

지침은 세션을 시작할 때 읽힙니다 — 규칙을 바꾼 뒤에는 codex를 다시 실행하고 “현재 적용된 지침을 요약해줘”로 확인합니다. AGENTS.md는 전역(~/.codex)·저장소 루트·하위 폴더 순으로 계층 병합됩니다.

.codex/config.toml — 승인 정책과 명령 규칙

권한 설정	무엇	언제
<code>-s read-only</code>	읽기 전용 — 분석·계획만	Review·스멜 탐지(9.1)
<code>-a untrusted</code>	상태 변경 명령마다 승인	기본값(권장 시작점)
Auto (on-request)	작업 폴더 안 편집·실행 자동	GREEN 반복이 익숙해진 후
<code>--yolo</code>	샌드박스·승인 해제(주의)	격리된 실습 저장소에서만

주의

본인이 쓰지 않은 AGENTS.md·config.toml·rules·스킬은 “신뢰할 수 없는 코드”처럼 다루세요. git push·배포·rm 같은 되돌리기 어려운 명령은 untrusted 모드에서 직접 승인합니다.

.codex/config.toml · rules (프로젝트 범위)

```
# .codex/config.toml — 신뢰한 프로젝트만 로드
approval_policy = "untrusted"
model_reasoning_effort = "medium"

# .codex/rules/default.rules
prefix_rule(pattern = ["git", "push"],
              decision = "prompt")
prefix_rule(pattern = ["rm", "-rf"],
              decision = "forbidden")
```

버전 메모

Codex는 명령 차단을 config.toml이 아닌 rules 파일(prefix_rule: allow·prompt·forbidden)로 관리합니다. 설정 키는 설치 버전의 공식 문서로 확인하세요.

커스텀 스킬 — ARRR를 \$호출 버튼으로

\$tdd-red

계약 ID의 '실패하는' 테스트만 작성. tests/만 수정, src/ 금지. 실패 로그 전체 표시. 첫 줄에 Phase: RED 선언.

\$tdd-green

실패 로그를 통과시키는 '최소' 구현만. 시그니처 불변 · 구현 줄에 ID 주석 · 테스트 수정/skip 금지.

\$tdd-refactor

동작·계약 불변으로 구조만 정리(중복 제거·SSOT). 전후 pytest·골든 게이트 확인. 없으면 "없음".

\$golden-capture

GREEN 상태의 동작을 입력 그리드 위에 스냅샷으로 떠 회귀 기준선. RED에서는 캡처 금지 — 버그 고정.

```
# .agents/skills/tdd-red/SKILL.md
--- name: tdd-red · description: TDD RED — 실패 테스트 먼저 ---
호출 시 지정한 계약의 실패 테스트만 tests/에 작성 ...
```

폴더명(name)이 곧 스킬 이름. \$tdd-refactor는 골든 게이트를 포함 — 리팩터 전후 `pytest -m golden -q`가 모두 GREEN 이어야 하고, 깨지면 되돌립니다(골든 수정은 금지 5조 위반).

확장 통로 — MCP · Hooks (설정은 부록 C)

MCP — 외부 도구 연결

config.toml에 [mcp_servers.이름]을 등록하면 context7(최신 라이브러리 문서)·github(이슈·PR) 같은 외부 도구를 세션에 연결한다. /mcp로 연결 상태와 도구 목록을 확인한다.

→ 설정 예시는 부록 C

Hooks — 감사 로그 자동 기록

도구 실행 전후에 스크립트를 걸어 “날짜 | 도구 | 사용자”를 감사 로그에 자동으로 남긴다. 스크립트에 실행 권한(chmod +x)이 없으면 조용히 실패한다.

→ 개념과 주의점은 부록 C

핵심

선택 심화 항목입니다 — 오늘 실습의 필수 경로는 아니지만, 실무 프로젝트에서 하네스를 확장하는 두 표준 통로입니다.

staging-spec 브랜치와 3종 커밋

터미널

```
# 1단계 — 계약 묶음 1개 = 브랜치 1개
$ git switch -c staging-spec/entity-cart

# 2단계 — RED(테스트만, 실패 확인 후 커밋)
$ git add tests/entity/test_cart.py
$ git commit -m "test(entity): INV-1 ... [RED]"

# 3단계 — GREEN(최소 구현만)
$ git add src/cart.py
$ git commit -m "feat(entity): INV-1 ... [GREEN]"
```

판단 기준 — 매번 이것만

새 계약(INV-*, E-*) 하나?
→ staging-spec/... 새 브랜치

테스트만? / 구현만? / 구조만?
→ [RED] / [GREEN] / [REFACTOR]

Entity+Boundary 동시?
→ git worktree로 폴더 분리

핵심

버전 이력 자체가 추적성 매트릭스의 시간 축 사본이 됩니다 — git log --oneline만으로 “어느 계약을, 어느 단계에서” 다뤘는지 추적됩니다.

4장 마무리 — 하네스 완성 후, 5장으로

- ☐ AGENTS.md가 존재하고 구조(ECB)·계약(C2C)·금지 5조항이 담겨 있다
- ☐ pytest.ini·conftest.py가 존재하고 pytest -q가 오류 없이 실행된다
- ☐ .codex/config.toml에 승인 정책이, .codex/rules/에 차단 명령이 설정돼 있다
- ☐ .agents/skills/*/SKILL.md로 \$tdd-red 등 커스텀 스킬이 동작한다
- ☐ “현재 지침을 요약해줘”로 AGENTS.md 적용을 확인했고, staging-spec/entity-cart 브랜치가 있다

원칙

4장에서 하네스(harness)를 세웠습니다 — 5장부터는 이 위에서 실제 제품 계약을 설계합니다.

05



CHAPTER 5

제품 계약 — 상류(Discovery)

Layer 1 불변식은 어디서 오는가? Mom Test로 고객 증거를 캐내 INV/E/AC + 증거 수준(L0~L2)으로 형식화합니다 — 사슬의 정점을 검증합니다.

🕒 4교시 · 상류(Discovery)

JTBD + Mom Test — 문제 발견의 한 쌍

J JTBD — 어떻게 구조화하나

그 답을 “사용자가 어떤 상황에서 무슨 일을 끝내려고 이 제품을 고용하는가(Job)”로 정리한다. Job이 계약의 주제가 된다.

M Mom Test — 무엇을 묻나

“이 제품 사실 거예요?”(거짓말 유발) 대신 과거의 실제 행동을 묻는 인터뷰 기법. 칭찬·가정·일반론을 걸러내고 사실만 남긴다.

1

Job 정리

상황 → 해야 할 일 → 기대 결과

2

인터뷰

Mom Test로 과거 행동·수치를 캐낸다

3

계약 변환

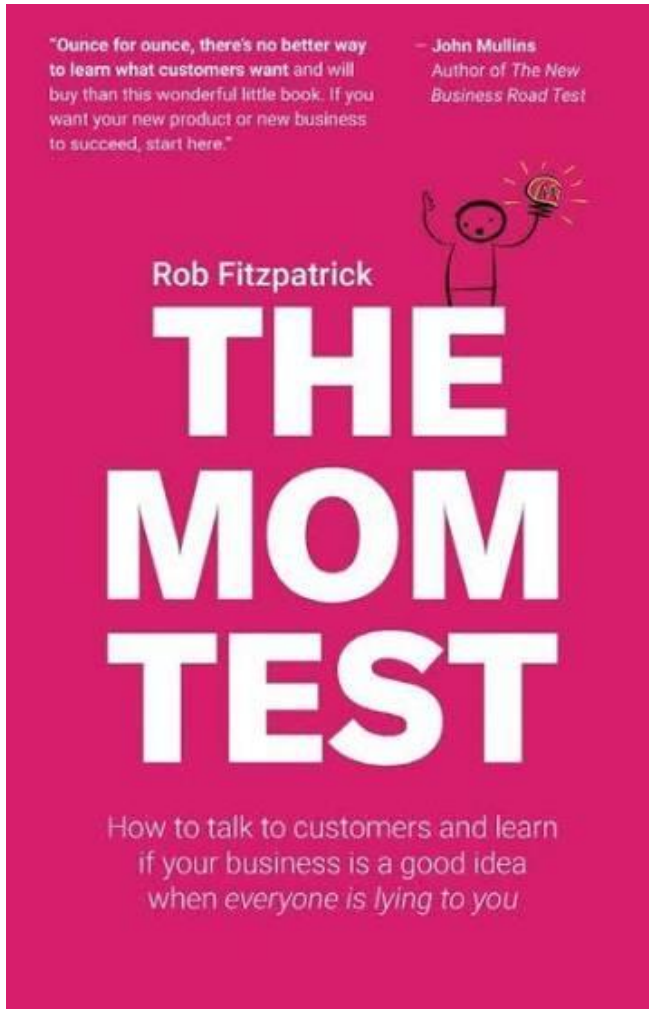
INV / E / AC + 증거 수준
L0~L2

4

PRD 고정

docs/PRD.md에 SSOT로 저장

The Mom Test란?



유래

롭 피츠패트릭(Rob Fitzpatrick)이 저술한 동명의 책에서 유래한 개념으로, 사업 아이디어가 유효한지 검증하기 위해 고객과 진솔한 대화를 나누는 방법론.

무엇을 막는가

상대가 '좋은 사람이라서' 해 주는 거짓 긍정(칭찬·공감)에 속지 않도록 질문하는 인터뷰 기법.

원칙

Mom Test = “사람의 말이 아니라 행동을 믿는 인터뷰 방법”. 계약(INV·E·AC)의 원천은 “고객이 실제로 어떻게 행동했는가”이며, Mom Test는 그 근거를 캐내는 도구다.

잘못된 질문 vs 올바른 질문

X 잘못된 질문

“제가 만든 설비 모니터링 앱 어때요?”

→ 칭찬만 들을 것

“이런 기능이 있으면 사용하시겠어요?”

→ 가짜 약속만 받을 것

“이 앱에 얼마 지불하시겠어요?”

→ 현실성 없는 답변만 받을 것

✓ 올바른 질문

“최근 설비 점검하며 가장 답답했던 순간은?”

→ 진짜 문제 발견

“지금은 그 상황을 어떻게 해결하고 계세요?”

→ 현재 우회 방법 파악

“그 방법의 어떤 점이 불편하세요?”

→ 개선 포인트 확인

핵심 원칙 3가지와 피해야 할 3가지

핵심 원칙 3가지

1 당신의 아이디어에 대해 말하지 마라

2 미래나 가정에 대해 묻지 마라

3 과거의 구체적인 행동에 대해 물어라

피해야 할 3가지 데이터

칭찬 (Compliments)

“정말 좋은 아이디어네요!” → 행동으로 이어지지 않음

가상의 약속 (Hypotheticals)

“있으면 꼭 쓸 것 같아요!” → 실제로는 안 씀

일반론 (Generics)

“보통 사람들은 좋아할 거예요” → 추측일 뿐

올바른 질문하기 — 과거 · 투자 · 해결 시도

1 과거의 구체적 사건

- “지난주 설비 점검 때 어떤 일이 있었나요?”
- “정확히 어떤 절차를 거치셨나요?”
- “얼마나 시간이 걸렸나요?”

2 돈·시간 투자 확인

- “현재 도구에 비용을 지불하시나요?”
- “배우는 데 시간을 얼마나 쓰셨나요?”
- “팀원들도 다 쓰고 있나요?”

3 해결 시도 이력

- “이 문제를 해결하려 시도해 보셨나요?”
- “어떤 도구들을 써보셨나요?”
- “왜 계속 쓰지 않으셨나요?”

인터뷰 진행하기 — Mom Test 실전 (Codex 역할극)

실무에서는 인터뷰 메모를 스스로 만들어야 한다. Codex에게 질문지를 짜게 하고, 고객 역할을 맡겨 연습한다.

▶ ① 인터뷰 가이드 생성

당신은 Mom Test 규칙을 엄격히 적용하는 인터뷰 설계자다

.

job: "주문 담당자가 할인 금액을 오류 없이 확정"

1. 과거 행동을 캐는 질문 8개 (미래·의견 금지)
2. 각 질문 옆에 '캐내려는 계약 후보' 한 줄
3. 반드시 포함:
'절대 발생하면 안 되는 결과는?' → INV 후보
'잘못 입력하면 뭐가 일어났나?' → E 후보
4. 물으면 안 되는 유도 질문 3개를 반례로

▶ ② 역할극 — Codex가 고객이 된다

지금부터 당신은 온라인 쇼핑몰 주문 담당자다.

- 과거에 실제 겪은 일만 말한다. 해결책을 제안하지 않는다.
 - 내가 유도 질문('~하면 되죠?')을 하면, 먼저 '유도 질문입니다'라고 지적하고 어떻게 바꿀지 알려준다.
 - '인터뷰 종료'라고 하면 [인터뷰 메모] 형식으로 정리한다. 없는 사실을 지어내지 않는다.
- 첫 질문을 기다린다.

인터뷰 메모를 계약으로 변환

[인터뷰 메모] — 실습 입력

- A 3개(12,000) + B 1개(30,000) = 66,000. 5만원 넘으면 10% 할인. 기대 59,400 (엑셀 계산 10분)
- VIP는 문턱 할인 후 추가 5%. 순서 문턱→VIP 고정 (순서 뒤바뀔 클레임 1건)
- 수량 -1 입력 시 합계 0원 표시(기대: 오류) / items 없이 제출 시 500 에러
- 쿠폰·배송비: 언급 없음 / “50,000원 정확히” 포함 여부 미확인

Discovery 트랙 — 6단계 (Codex에 지시)

Step 1 각 항목을 증거 수준(L0/L1/L2)으로 분류

Step 2 Invariant / Error / AC / Out-of-Scope 추출

Step 3 ID 부여 (INV-n · E-n · AC-n · OOS-n)

Step 4 테스트 가능한 문장으로 변환

Step 5 AC는 Given-When-Then으로

Step 6 출력: ID | 계약 | 증거 수준 | 계층 | 테스트 아이디어

규칙: 근거 없는 동작을 계약 ID로 만들지 마라.

확정 계약표를 docs/PRD.md에 고정 (SSOT)

ID	계약 (불변식 / 에러)	증거 수준	계층
INV-1	subtotal(items) == $\Sigma(\text{price} \times \text{qty})$	— (수학적 정의)	Entity
INV-2	amount $\geq 50000 \rightarrow \text{round}(\times 0.9)$ / 미만은 그대로 (경계 포함)	L1	Entity
INV-3	문턱 할인 후 VIP면 $\text{round}(\times 0.95)$. 순서 문턱→VIP 고정	L1	Entity
INV-4	모든 입력에서 $0 \leq \text{final_total} \leq \text{subtotal}$	L0	Entity
E-1	items is None $\rightarrow$ TypeError	L0	Boundary*
E-2	price 또는 qty가 음수 $\rightarrow$ ValueError	L0	Boundary*

* E-1·E-2는 입력 검증 성격이지만 순수 함수 subtotal()의 가드로 구현합니다.

핵심

docs/PRD.md = SSOT(단일 진실). 계약표 + AC(Given-When-Then) + 구현 금지(OOS: 쿠폰·세금·배송비·포인트)를 포함한다. 아직 “구현 완료”라 쓰지 않는다 — 현재는 RED 준비.

06



CHAPTER 6

Track B — Entity/Logic

장바구니 계산 로직(순수 함수)을 8단계 골격으로 완주합니다. INV-1~4 · E-1~2를 ARRR 사이클로 하나씩 세우고, 골든 마스터로 리팩터를 보호합니다.

🕒 5교시 · Track B 실습

Discovery ①②③ — 목표 · 실패 조건 · 계약

① 목표 좌표 — 무엇을 보장하는가

- 보장: 합계 → 문턱 할인 → VIP 할인을 정확·일관되게 계산
- ECB: Entity 계층 — 순수 로직, 외부 I/O·UI 문자열 없음
- C2C: Layer 4 구현 단위 + Layer 1 불변식
- 트랙: Track B — Domain/Logic (tests/entity/)

② PRD — “무엇이 실패인가” (L0~L3)

- L0 계약 위반 — None인데 0 반환 → 예외를 던져야 (E-1)
- L1 경계 — 정확히 50,000인데 할인 안 됨 → “이상($\geq$)” 위반 (INV-2)
- L2 도메인 — VIP인데 5% 빠지거나 순서가 뒤바뀜 (INV-3)
- L3 수학적 불변 — 최종 금액이 subtotal보다 커진다 (INV-4)

③ 계약 확인 — 대상 함수 3개 (충족할 계약: INV-1~4 · E-1~2)

```
def subtotal(items): ...           # INV-1, E-1, E-2 — 시그니처는 여기서 고정된다
def apply_threshold_discount(amount): ... # INV-2
def final_total(items, is_vip=False): ... # INV-3, INV-4
```

테스트 플랜 — RED 전에 테스트 이름을 못 박는다

계약	테스트 이름	유형	핵심 검증
INV-1	test_subtotal_sums_price_times_qty	예제	합계 = $\sum \text{price} \times \text{qty}$
INV-2	test_threshold_applies_at_50000	경계	50000 적용 · 49999 미적용
INV-3	test_vip_order_and_value	순서	문턱→VIP, 60000→51300
INV-4	test_discount_never_increases	속성	$0 \leq \text{final} \leq \text{subtotal}$
E-1 / E-2	test_none_raises / test_negative_raises	예외	TypeError / ValueError

핵심

5종 테스트 유형(예제·경계·순서·속성·예외)이 곧 Right-BICEP·CORRECT의 실전판입니다 — 특히 INV-4는 여러 입력을 도는 “속성 테스트”로, Layer 1 불변식의 본래 형태입니다.

④ 스켈레톤 — 시그니처 + TODO (완성 코드는 감춤)

src/cart.py

```
# src/cart.py — 스켈레톤 (의도적 미구현)
THRESHOLD = 50000      # INV-2 문턱 금액 (SSOT)

def subtotal(items):
    raise NotImplementedError # TODO: INV-1, E-1, E-2

def apply_threshold_discount(amount):
    raise NotImplementedError # TODO: INV-2

def final_total(items, is_vip=False):
    raise NotImplementedError # TODO: INV-3, INV-4
```

▶ 스켈레톤 프롬프트

@docs/PRD.md 를 읽고 src/cart.py 에 스켈레톤 만 만들어줘.

- 함수 3개의 시그니처 고정
- 본문은 raise NotImplementedError
- TODO 주석에 계약 ID
- 완성 로직·테스트는 쓰지 마

원칙

완성 코드를 감추는 것은 교육 장치이자 추적성 장치 — 학습자와 AI 모두 “계약 → 실패 테스트(RED) → 최소 구현(GREEN)” 순서로 만 코드에 도달합니다.

subtotal — INV-1 · E-1 · E-2

RED — 실패 테스트 먼저

```
# RED — tests/entity/test_cart.py
def test_subtotal_sums():          # INV-1
    items = [{'price': 12000, 'qty': 3},
              {'price': 30000, 'qty': 1}]
    assert subtotal(items) == 66000

def test_none_raises():           # E-1
    with pytest.raises(TypeError):
        subtotal(None)
```

GREEN — 최소 구현, 줄마다 ID

```
# GREEN — src/cart.py
def subtotal(items):
    if items is None:              # E-1
        raise TypeError('items is None')
    for i, it in enumerate(items): # E-2
        if it['price'] < 0 or it['qty'] < 0:
            raise ValueError(f'negative at {i}')
    return sum(it['price'] * it['qty'] # INV-1
               for it in items)
```

핵심

실패 로그를 Context로 준 뒤 “최소 구현만”을 요청 — 구현 줄마다 어떤 계약(INV-1 · E-1 · E-2)을 지키는지 주석으로 답니다.

apply_threshold_discount — INV-2, 경계 ≥ 50000

RED

```
# RED — 경계 2행이 핵심
def test_threshold_at_50000():    # INV-2
    assert apply_threshold_discount(50000) == 45000
    # 이상(>=) 적용

def test_below_threshold():
    assert apply_threshold_discount(49999) == 49999
    # 미만은 그대로
```

GREEN

```
# GREEN — src/cart.py
THRESHOLD = 50000           # INV-2 SSOT
THRESHOLD_RATE = 0.9       # INV-2 SSOT

def apply_threshold_discount(amount):
    if amount >= THRESHOLD:    # INV-2
        return round(amount * THRESHOLD_RATE)
    return amount
```

주의

경계값 50000 → 45000 — INV-2의 “이상($\geq$)”이 코드에서 “초과($>$)”로 미끄러지는 사고가 가장 흔합니다. 매직 넘버 $50000 \cdot 0.9$ 는 즉시 상수(SSOT)로 추출합니다.

final_total — INV-3 순서 · INV-4 속성

RED — 순서 + 속성

```
# RED
def test_vip_order_and_value(): # INV-3
    items = [{'price': 60000, 'qty': 1}]
    assert final_total(items, True) == 51300
    # 60000→54000(문턱)→51300(VIP)

def test_never_increases(): # INV-4
    for items, vip in samples:
        total = final_total(items, vip)
        assert 0 <= total <= subtotal(items)
```

GREEN — 재사용이 곧 보장

```
# GREEN
VIP_RATE = 0.95 # INV-3 SSOT

def final_total(items, is_vip=False):
    amount = apply_threshold_discount(
        subtotal(items)) # INV-3 문턱 먼저
    if is_vip: # INV-3 그다음
        amount = round(amount * VIP_RATE)
    return amount # INV-4 할인만 적용
```

핵심

이미 만든 subtotal · apply_threshold_discount를 재사용하라고 명시하면 중복 없이 통과하고, 재사용 자체가 INV-4(할인만 적용 → 금액이 늘 수 없음)를 자연스럽게 보장합니다.

리팩터 전 — 골든 마스터로 회귀 기준선을 뜯다

규율 ① — GREEN 상태에서만 캡처 (tests/golden/cart_golden.json)

```
{ "apply_threshold_discount": [
  {"in": 49999, "out": 49999},
  {"in": 50000, "out": 45000},
  {"in": 60000, "out": 54000},
  {"in": 66000, "out": 59400} ] }
```

규율 ② — 골든 파일을 고쳐 통과시키지 않는다

리팩터 후 골든이 깨지자 골든을 재생성해 통과시키는 것은 “테스트를 바꿔 통과시키기”와 같은 회피 → AGENTS.md 금지 5조. 계약이 실제로 늘었다면 chore(golden): ... 커밋으로 의도적 재생성을 이력에 남깁니다.

주의

판정 기준 — 골든 diff가 0이 아닌 refactor 커밋은 그 자체로 규칙 위반입니다. RED 상태에서 캡처하면 버그를 기준으로 고정시킴으로, 반드시 `pytest -q` 통과 뒤에 캡처합니다.

완성 코드와 C2C 추적성 매트릭스

src/cart.py — 완성

```
THRESHOLD      = 50000 # INV-2 (SSOT)
THRESHOLD_RATE = 0.9   # INV-2 (SSOT)
VIP_RATE       = 0.95  # INV-3 (SSOT)

def subtotal(items):
    if items is None: raise TypeError(...) # E-1
    for i, it in enumerate(items): ...     # E-2
    return sum(it['price'] * it['qty'] ...) # INV-1

def final_total(items, is_vip=False):
    amount = apply_threshold_discount(...) # INV-3
    if is_vip: amount = round(amount*VIP_RATE) # INV-3
    return amount                          # INV-4
```

▶ 매트릭스를 Codex에게 검사시킨다

@docs/PRD.md @tests/ @src/ 계약표와 테스트·구현을 대조해 추적성 매트릭스를 만들어라. 별도 보고:

① 빈칸 — 계약은 있는데 테스트/구현이 없는 행

② 계약 ID 주석 없는 “정체불명” 코드 줄

파일은 수정하지 마라. docs/traceability.md로 저장만.

```
$ pytest -q → 6 passed in 0.07s ✓
```

핵심

모든 구현 줄에 계약 ID가 달려 “이 줄이 어느 계약을 지키는가”가 코드만 봐도 읽힙니다 — 표를 수작업으로 채우지 말고 Codex에게 검사시키는 것이 Review 트랙의 표준입니다.

6장 정리 — Track B(Entity) 사이클 완주

☐ pytest -q가 6 passed로 끝난다 — subtotal · apply_threshold_discount · final_total 구현 완료

☐ 구현 줄마다 INV-1~4 · E-1~2 계약 ID 주석이 달려 있다

☐ test(RED) → feat(GREEN) → refactor(REFACTOR) 3종 커밋이 ID와 함께 분리돼 있다

☐ 골든 마스터가 GREEN 상태에서 캡처됐고, 리팩터 후에도 diff가 0이다

☐ docs/traceability.md에 계약↔테스트↔구현 매트릭스가 저장돼 있고 빈칸이 없다

원칙

Track B(Entity) 전체 사이클을 완주했습니다 — 7장에서는 Track A(Boundary)로 넘어가, 이 final_total을 감싸는 Flask 웹 폼에서 계약이 어떻게 검증되는지 살펴봅니다.

07



CHAPTER 7

Track A — UI/Boundary

Entity 로직을 감싸는 Flask 주문 폼을 세웁니다. Boundary는 Entity를 재사용하되 재계산하지 않고, 계약은 UC-1 · UC-2 · UE-1로 검증됩니다.

🕒 6교시 · Track A 실습

목표 · UI 계약 · 테스트 플랜

① 목표 좌표 — ECB 방향 단속

- 보장: 주문 폼에서 수량·VIP를 입력받아 최종 금액을 정확히 표시
- ECB: Boundary — 입력 검증·출력 형식화 (계산은 Entity에 위임)
- Boundary → Entity ✓ 허용 (from src.cart import final_total)
- Entity → Boundary ✗ 절대 금지 (cart.py는 Flask 미import)

ID	계약	실패 시나리오
UC-1	GET / 는 200 + 수량·VIP 폼 포함	L1 폼이 안 뜸
UC-2	POST /calc → final_total 결과 표시	L2 금액 오표시
UE-1	qty가 숫자가 아니면 400	L0 서버 오류

플랜

테스트 플랜 — UC-1 test_form_renders · UC-2 test_calc_shows_final_total · UE-1 test_calc_rejects_non_numeric_qty.
계산 자체는 Entity의 final_total이 이미 INV-1~4로 보장 — Boundary 테스트는 “올바른 함수를 부르고 결과를 형식에 맞게 돌려주나”를 검증합니다.

④ 스켈레톤 — 좋은 프롬프트와 나쁜 프롬프트

▶ app.py 스켈레톤 (좋은 예)

```
## 수정 허용: src/app.py만. tests/·cart.py 금지
## 계약(docs/PRD.md): UC-1 · UC-2 · UE-1
## ECB: from src.cart import final_total만 허용
## 할인 공식·문턱·VIP 순서를 재구현하지 않는다
## 산출물: @app.get('/'), @app.post('/calc'),
## 핸들러 본문 raise NotImplementedError, TODO에 계약 ID
## 금지: 구현 로직, HTML 상세, DB, 세션, 쿠폰
## 완료: import 가능, 호출 시 NotImplementedError
```

✗ 나쁜 프롬프트 — 왜 나쁜가

“Flask 주문 품을 완성해줘”

→ 범위·필드·응답 빠짐

“할인 로직도 테스트해줘”

→ INV는 Entity 책임, 역할 침범

“예쁜 UI로”

→ ID 없는 동작 = 과잉 구현

RED와 GREEN을 한 프롬프트에

→ 순서 위반

“음수 qty도 400으로”

→ UE-1은 비슷자만, 음수는 E-2(Entity)

원칙

스켈레톤 프롬프트에는 “단계 한 줄 · 수정 허용 파일 · 계약 ID 표 · ECB 한 줄 · 완료 기준” 다섯 가지를 반드시 넣습니다.

ARRR — Flask test_client로 RED → GREEN → REFACTOR

RED

```
# tests/boundary/test_app.py (RED)
from src.app import app

def test_form_renders():          # UC-1
    res = app.test_client().get('/')
    assert res.status_code == 200
    assert b'name="qty"' in res.data

def test_calc_shows_final_total(): # UC-2
    res = app.test_client().post('/calc', data=
        {'price': 60000, 'qty': 1, 'vip': 'on'})
    assert b'51300' in res.data
```

GREEN

```
# src/app.py (GREEN)
from flask import Flask, request
from src.cart import final_total # B→E
app = Flask(__name__)

@app.post('/calc')
def calc():
    try:                                # UE-1
        qty = int(request.form['qty'])
    except (KeyError, ValueError):
        return '숫자여야 합니다', 400
    total = final_total(...)           # UC-2 재사용
```

핵심

브라우저 없이도 HTTP 동작을 검증하는 test_client가 Boundary 트랙의 하네스입니다 — 외부 의존을 경계 뒤에 격리한 덕분.
 REFACTOR: 폼 HTML이 길어지면 템플릿으로 분리(Extract)가 자연스러운 다음 정리.

브라우저로 시각 확인 — 사람이 눈으로 보는 계약

flask --app src/app run 실행 후 `http://127.0.0.1:5000` 접속 → 값을 바꿔 가며 확인. `test_client`가 계약을 보장하지만, 계약이 “사용자가 실제 겪는 것”과 일치하는지는 사람이 한 번 봐야 합니다.

가격	수량	VIP	기대
60000	1	✓ (체크)	51300
60000	1	—	54000
50000	1	—	45000
49999	1	—	49999
60000	abc	—	400 에러

✓ 완료 신호

- `pytest -q` → 9 passed (6장 6 + 7장 3)
- `cart.py`에 `from flask` 없음
- `app.py`가 `final_total`을 재사용

주의

경계값 50000 → 45000이 나오는지 반드시 눈으로 확인하세요 — INV-2의 “이상($\geq$)”이 코드에서 “초과($>$)”로 미끄러지는 사고가 가장 흔합니다.



PART Ⅲ

적용 과제 · 마무리 — 새 도메인에 적용하기

같은 C2C × Dual-Track TDD 리듬을 새 도메인(Magic Square)에 적용하고, Review · KPT 회고로 마무리합니다. Codex CLI + VS Code · Python 3 · pytest

08



CHAPTER 8

Magic Square 4×4 — 관통 프로젝트

Review assignment · QA 프롬프팅. 계약 → PRD → 하네스 → ARRR → 기능 확장 → 보고서까지, 하루를 하나로 잇는 예제입니다. Codex CLI + VS Code · Python 3 · pytest · pytest-cov

🕒 7교시 · 적용 과제 ①

Magic Square 4×4 — 하루를 하나로 잇는 예제

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

모든 행·열·대각선의 합 = 34

검증 라인 10개: R1~R4 · C1~C4 · D1 · D2

도메인

- 4×4 · 채운 칸 1~16 · 빈칸 0 · 마법상수 34
- 검증 대상 10선: R1~R4 · C1~C4 · D1 · D2
- 목표 = 풀기가 아님 → 어느 줄이 왜 틀렸는지 10초 안에
- 자동 풀이·채우기 = OOS(범위 밖)

```
validate_lines(grid) -> {status, failed_lines}
```

원칙

7단계 = 8단계 골격 + 13 STEP의 실행 · ③~⑥이 ARRR = C2C × Dual-Track TDD

프로젝트 초기화 — MagicSquare

(예: MagicSquare). 오늘 하루 이 폴더 하나로 끝까지 갑니다. 폴더를 만든 뒤 세션을 시작합니다.

① 프로젝트 골격 (폴더 · venv · 기본 디렉터리)

```
$ cd C:/DEV # (macOS/Linux: ~/dev)
$ mkdir MagicSquare && cd MagicSquare
$ git init
$ python -m venv .venv
$ source .venv/Scripts/activate # macOS/Linux: .venv/bin/activate
$ pip install pytest pytest-cov
$ mkdir -p src tests docs Report Prompting .agents/skills
```

② 세션 시작

```
$ codex
# MagicSquare 폴더에서 실행
```

폴더 역할

- docs/ — PRD · README
- Report/ — 작업 보고서
- Prompting/ — 대화록
- .agents/skills/ — 스킬

브랜치 전략 — staging에서 갈라 staging으로 모은다

브랜치 구조 (staging의 형제 브랜치들)

```
main
├── staging      # 통합 브랜치 (검증된 것만 모임)
│   ├── spec    # 계약·PRD·하네스
│   ├── red     # Ask (실패 테스트)
│   ├── green   # Respond (최소 구현)
│   ├── refactoring # Refine
│   └── new_features # STEP 7: 확장
```

각 브랜치: staging에서 분기 → 작업·커밋 → staging으로 머지

시작 — staging과 spec

```
$ git switch -c staging
# main → staging (통합 브랜치)
$ git switch staging
$ git switch -c spec
# staging → spec (계약·PRD·하네스)
# ... 작업·커밋 ...
$ git switch staging
$ git merge --no-ff spec
```

핵심

각 단계(spec · red · green · refactoring · new_features)를 staging에서 갈라 작업한 뒤 다시 staging으로 머지합니다. 커밋 메시지에 계약 ID를 적으면 git log만으로 “어느 계약이 어느 단계에 있는지”가 추적됩니다. (staging → main 최종 머지는 과정 범위 밖)

발견 — Mom Test 인터뷰와 계약 변환 (증거 수준 L0~L2)

▶ Mom Test 인터뷰 (파일 수정 없음)

너는 Mom Test 원칙을 지키는 인터뷰어(Rob Fitzpatrick 스타일)야.
규칙: (1) 내 아이디어를 말하지 않는다 (2) 과거·구체·사실만 묻는다 (3) 적게 말하고 많이 듣는다.
금지: "만들면 좋겠어?", "이 기능 필요해?", "TDD 하면 어때?" 같은 유도/미래 질문.
먼저 인터뷰 질문 1번만 해. 솔루션 언급 금지.

▶ 계약표 생성

[발견 → 계약] Mom Test 결과:
- 페르소나: [...] - 진짜 문제(한 문장): [...] - 증거 3줄: [...]
MagicSquare의 제품 계약표를 만들어줘.
출력: |ID|계약|증거 수준(L0~L2)|계층/트랙|검증 테스트 ID| 표 + OOS 표.
ID 체계: 불변식 INV-*, 에러 계약 E-*, UI 계약 UC-*.
인터뷰 사실에 없는 기능을 임의로 추가하지 마(과잉 명세 금지). 파일은 수정하지 마.

증거 수준	L0	L1	L2
의미	협상 불가능한 외부 제약 또는 수치를 동반한 과거 행동	고객 발화 · 실측 근거	추정 (추가 검증 대상)

PRD(SSOT) 고정과 Harness 골격 — Agent 모드

▶ PRD(SSOT) 고정

docs/PRD.md 를 생성해줘. 오늘의 SSOT다.
 포함: 진짜 문제 한 문장 · R-G-I-O · 확정 계약표(INV/E/UC) ·
 OOS · 골든셋 G1~G5 · 성공 기준.
 다른 파일은 만들지 마.

▶ Harness 골격 생성

MagicSquare 용 최소 Harness만 만들어줘. 본문은 비워.
 - pyproject.toml : pytest 만 설정
 - src/__init__.py : 빈 파일 - src/validate_lines.py : 함수 시그니처만
 - tests/__init__.py : 빈 파일 - tests/test_validate_lines.py : import 줄만
 구현·테스트 본문은 아직 쓰지 마. 골격만.

▶ AGENTS.md 생성

루트에 AGENTS.md 초안을 만들어줘. 40~60줄. 다른 파일은 만들지 마.
 도메인: 4×4, 빈칸 0, 1~16, 마법상수 34, 10선(R1~R4 · C1~C4 · D1 · D2)
 API: validate_lines(grid) -> {status: pass|fail|incomplete, failed_lines: [...]}
 TDD: RED→GREEN→REFACTOR, RED는 tests/만 수정. 금지 5조항과 계약 ID 규율 포함.
 AI: 한국어, TDD 시 첫 줄 Phase 선언, git commit은 사용자 요청 시만

README — PRD 참조 RED To-Do 체크리스트와 릴리즈 정보

▶ README ① — RED To-Do 체크리스트

docs/PRD.md 를 읽고 docs/README.md 를 만들어줘.

README 상단에 "ASK(RED) To-Do"를 체크리스트로 담는다:

- 계약 ID(INV-*, E-*, UC-*)마다 한 줄씩, 형식:

"- [] <ID> — <검증 테스트 ID>: <한 문장 테스트 아이디어> (파일: tests/..., Track: B 또는 A)"

- RED 원칙 한 줄 명시: 구현 없이 실패 테스트만 · tests/만 수정 · 통과시키지 않는다.

- 골든셋 G1~G5도 "- [] G# — <판> → <기대>" 형식의 체크 항목으로 넣는다.

PRD.md에 없는 계약은 추가하지 마(과잉 명세·SSOT 위반 금지). 그 외 파일은 만들지 마.

▶ README ② — 릴리즈 정보 추가

[이어서] 같은 docs/README.md 맨 아래에 "릴리즈 정보" 섹션을 추가해줘.

- 버전(SemVer, 예: v0.1.0) · 날짜 · 이번 사이클 범위(구현할 계약 ID 목록).

- OOS(이번에 안 할 것) 요약 한 줄 (PRD.md의 OOS 인용).

- 완료 기준(DoD): 골든셋 G1~G5 GREEN · pytest 전부 GREEN · 추적성 매트릭스 빈칸 없음.

- 변경 이력(Changelog) 표: | 버전 | 날짜 | 요약 |, 첫 행에 이번 릴리즈.

PRD.md의 사실만 사용하고, 없는 값은 비워 두되 추측하지 마.

첫 커스텀 스킬 \$tdd-red — 하네스가 살아있는가

.agents/skills/tdd-red/SKILL.md

```

---
name: tdd-red
description: TDD RED — 실패 테스트 먼저 (validate_lines)
---
validate_lines 의 새 동작을 검증하는 실패 테스트를 tests/ 에만 작성한다.
## 필수 선언 (응답 첫 줄)
Phase: red | Target: validate_lines | Track: logic
## 절차
1. 케이스의 입력 grid 와 기대 출력(status, failed_lines)을 확정한다.
2. tests/test_validate_lines.py 에 AAA 구조 테스트 1개를 추가한다.
3. pytest 로 FAIL 을 확인한다 (구현이 없으므로 실패가 정상).
4. 통과시키려 하지 않는다. RED 는 실패가 목표다.
## 보고: 테스트 함수명 · 입력/기대 · pytest FAIL 한 줄 · 변경 파일
## 금지: src/ 수정 · assert 완화/skip/xfail · 한 번에 여러 케이스
## 입력: $tdd-red 뒤에 적은 요청이 이번 케이스다

```

▶ 첫 RED (그대로 붙여 넣기)

\$tdd-red D-VAL-02 [INV-2]: 완성 격자에서 (2,2)=6 을 7 로 바꿔 R2·C2 가 35가 되는 fail 케이스.
기대: status="fail", failed_lines 에 "R2"와 "C2" 포함. src/ 수정 금지.

✓ 기대 결과

첫 줄에 Phase 선언 · pytest 1 failed · 변경 파일은 tests/ 뿐

실패하는 테스트 먼저 — \$red-test-plan

예제로 D-LOC-01 [INV-5](빈칸 좌표를 1-index · row-major로 반환)를 잡습니다. 먼저 red 브랜치를 staging에서 가릅니다.

red 브랜치 분기

```
$ git switch staging && git switch -c red
```

커스텀 스킬 세트

- \$red-test-plan · \$red-skeleton
- \$green-minimal · \$golden-master
- \$refactor-smell · \$refactor-safe
- 모두 .agents/skills/*/SKILL.md — 8.5의 \$tdd-red를 단계별로 세분화한 것

▶ \$red-test-plan (/plan 모드)

\$red-test-plan

Phase: red | Layer: entity | Track: Logic

이번 RED 묶음: D-LOC-01 [INV-5]

1. C2C 추적: docs/PRD.md 의 INV-5 인용 → To-Do 1개 → Given/When/Then
2. Track B 설계표 | D-LOC-01 | find_blank_coords | G1→[(2,3),(4,4)] | INV-5
|
3. 테스트 플랜: tests/entity/test_d_loc_01.py · conftest 픽스처 grid_g1
4. ECB·Mock 점검: Logic Track → Domain Mock 금지, entity E001~E005 emit 금지
금지: src/ 수정, GREEN/REFACTOR, skip/xfail. 완료 후 "\$red-skeleton 준비됨" 한 줄.

RED 스켈레톤 — \$red-skeleton

▶ \$red-skeleton (Agent 모드)

```
$red-skeleton
Phase: red | Layer: entity | Track: Logic | Test ID: D-LOC-01
[INV-5]
파일: tests/entity/test_d_loc_01.py · 픽스처: tests/conftest.py
(grid_g1 — 0이 2개)
규칙: AAA 주식 / Then은 pytest.fail("RED: D-LOC-01 — ...") 한 줄만
/
assert 본문·skip·xfail 금지 / src/ 수정 금지 / 상수 34·16·4는
constants.py import
완료 후 pytest 실행하고 보고: Test ID · FAIL 한 줄 · 변경 파일(tests/
만)
```

생성 결과 — tests/entity/test_d_loc_01.py

```
import pytest

def test_d_loc_01_blank_coords(grid_g1):
    """D-LOC-01 [INV-5] 빈칸 좌표는
    1-index · row-major 순서"""
    # Given — grid_g1: 빈칸(0) 2개
    # When — find_blank_coords(grid_g1)
    # Then
    pytest.fail("RED: D-LOC-01 — 미구현")
```

핵심

스켈레톤 단계의 RED는 `pytest.fail`로 “의도된 실패”를 명시합니다 — 기대값(`assert`)은 GREEN 단계에서 실제 `assert`로 교체합니다 .

통과하는 최소 구현 — \$green-minimal · 골든 마스터

green 브랜치 분기

```
$ git switch staging && git switch -c green
```

▶ \$green-minimal

\$green-minimal

Phase: green | Layer: entity | Track: Logic | RED 대상: D-LOC-01 [INV-5]

1. RED 재확인 → 2. src/entity/ 에 최소 구현 (구현 줄 [INV-5] 주석, 매직넘버 금지 → constants.py SSOT, E001~E005 raise 금지, entity 는 boundary/control import 금지)
3. pytest.fail 제거 → 실제 assert 로 교체 → 4. PASS 확인
금지: 이번 묶음 외 ID 동시 해결, REFACTOR, assert 완화. git commit은 내가 요청할 때만.

Golden Master — 출력 계약 고정

확정한 출력을 기준 파일로 고정해, 이후 리팩터·확장이 출력을 몰래 바꾸지 못하게 하는 회귀 안전망(Approval Test). 전제: 대상 Test ID 가 PASS.

▶ \$golden-master

\$golden-master

Phase: green | Layer: entity | Track: Logic

대상: validate_lines 출력(G1~G3)을 tests/golden/*.approved.txt 로 고정.

tests/_approval.py 헬퍼 생성. 이후 출력이 바뀌면 테스트가 FAIL 하도록.

동작 불변, 구조만 — 탐지와 실행을 분리한다

확장(기능)과 수축(정리)의 균형 — 확장 2~3회마다 수축 한 번이 '씨앗을 먹지 않는' 리듬입니다. 탐지(Ask)와 실행을 분리하고 한 번에 하나만.

refactoring 브랜치 분기

```
$ git switch staging && git switch -c refactoring
```

▶ \$refactor-smell (/plan — 수정 금지)

```
$refactor-smell
```

Phase: refactor | Scope: src/ tests/ | Track: Logic+UI

전제: pytest 전부 PASS — 아니면 중단. 스멜 표로 스캔만(코드 수정
·commit 금지):

| 우선순위 | 스멜 | 위치(파일:함수) | 근거 | Budget 내 후보 |

Change Budget: 파일≤3 · 클래스≤1 · 메서드≤3.

출력: P0/P1/P2 + 후보 1~3개, P0 1개만 실행 안내.

▶ \$refactor-safe (스멜 1개만)

```
$refactor-safe
```

Phase: refactor | Layer: entity | Track: Logic

스멜 1개 (P0): Duplicated Code — 10선 합 계산 4회 반복

대상: src/validate_lines.py:validate_lines

작업: _collect_failed_line_ids(grid) 헬퍼 추출

원칙: 입출력·failed_lines ID·순서 변경 금지 [INV-2·INV-4] · assert 완
화·기능 추가 금지

완료 후: pytest 전부 PASS + golden matched 확인

스멜 표의 P0 하나만 실행 → pytest + golden 확인 → 커밋 → 다음 스
멜. 한 커밋에 스멜 하나.

다음 계약으로 — 같은 리듬을 계약마다 반복

1

red

staging에서 분기 → 계약 하나의 실패 테스트

2

green

최소 구현 → 실제 assert로 교체 → PASS

3

refactoring

스멜 1개만 → pytest + golden 확인

4

staging 머지

--no-ff 머지 → 다음 계약 ID로

Track B — validate_lines

D-VAL-01~04 [INV-1~4]를 이어 갑니다: red → green → refactoring 브랜치를 staging에서 갈라 머지, 한 번에 계약 하나씩. Domain Mock 금지.

Track A — 입력 계층

U-IN-01/02 [E-1/E-2]도 Layer만 boundary로 바꿔 동일 패턴. Track A는 도메인을 mock으로 격리해 UI 계약만 검증합니다.

기능 확장 — Track A GUI

검증 로직(Track B)이 계약대로 완성되면, 같은 계약 위에 얇은 UI(Track A)를 올립니다. UC-1은 4×4 격자와 [검증] 버튼을 표시하고, 눌렀을 때 `status · failed_lines`를 화면에 보여 줍니다.

`new_features` 브랜치 분기

```
$ git switch staging && git switch -c new_features
```

Track A의 규율

- `boundary` 계층에서만 에러 코드(E002 · E003)를 처리합니다 — `entity`는 에러 코드를 모릅니다(ECB).
- Track A에서는 도메인(`validate_lines`)을 `mock`으로 격리해 UI 계약(U-GRID-01 · U-VAL-01)만 테스트합니다.

▶ Track A RED (Layer만 boundary로)

```
$red-test-plan
```

```
Phase: red | Layer: boundary | Track: UI
```

```
이번 RED 묶음: U-IN-01 [E-1], U-IN-02 [E-2]
```

```
표: 입력 계층 검증 —
```

```
grid=None → E003
```

```
셀 값이 1~16 밖 → E002
```

```
validate_lines는 mock.
```

문서 자동 생성 — 작업 보고서 8섹션

1 문제·목표 (R-G-I-O)

2 확정 계약표

3 골든셋

4 ARRR 진행 로그 (계약 ID별 RED/GREEN/REFACTOR)

5 추적성 매트릭스

6 리팩터 기록

7 남은 OOS · 다음 사이클 후보

8 KPT 회고

▶ 보고서 생성 (Agent)

Report/ 의 다음 번호로 NN.REPORT.md 를 만들어줘. 위 8섹션 구조.
docs/PRD.md · git log · pytest 결과를 인용해 채우고, 없는 사실은 추측하지 마.
함께 Prompting/NN.PROMPTS.md 에 오늘의 핵심 프롬프트를 정리해줘.

핵심

Report/NN.REPORT.md에는 8섹션을, Prompting/NN.*에는 대화록을 남깁니다 — 보고서도 “근거 없는 내용 금지” 규칙을 그대로 따릅니다.

09



CHAPTER 9

마무리 — Review · KPT 회고 · 현업 적용

커밋 이력으로 추적성을 확인하고, 수정 없는 Review로 감리한 뒤, KPT로 오늘을 돌아보고 팀에 가져갈 액션을 정합니다.

🕒 마무리

커밋 추적과 Review — 수정 없이 점검 (읽기 전용)

형식: <타입>(<계층>): <계약 ID> <무엇> [단계]

```
$ git log --oneline
a1b2c3d refactor: VIP_RATE SSOT 추출 [REFACTOR]
9f8e7d6 feat: INV-3.4 final_total [GREEN]
7c6b5a4 test: INV-3.4 실패 테스트 [RED]
3b2a1c0 feat: INV-2 문턱 할인 구현 [GREEN]
1a0b9c8 test: INV-2 실패 테스트 [RED]
# 이력 자체가 추적성 매트릭스의 시간 축 사본
```

Review — codex --sandbox read-only

- 1 구현 줄의 계약 ID 주석 누락
- 2 매트릭스의 빈칸
- 3 과잉 구현 · 우회 (assert True / skip)
- 4 구조·동작이 한 커밋에 섞였나 (Tidy First)
- 5 Entity가 Boundary를 import하나 (ECB)

원칙

머지 전 감리 — 사람이 규칙을 외워 검사하는 것이 아니라, 규칙을 프롬프트로 고정해 AI에게 검사시키는 것이 Review 트랙입니다. 발견 사항만 목록으로 보고하고 파일은 고치지 않습니다.

KPT 회고 & 현업 적용 — 팀 액션 아이템

Keep (계속할 것)	Problem (문제)	Try (시도할 것)
계약 → 테스트 → 구현 순서		
한 번에 계약 하나씩		
커밋 메시지에 계약 ID		

팀 액션 아이템

- 1 우리 팀의 '진짜 문제'를 Mom Test로 다시 확인한다 (솔루션 먼저 말하지 않기).
- 2 핵심 규칙을 AGENTS.md로 고정하고, 금지 5조항을 팀 표준으로 삼는다.
- 3 모든 PR을 계약 ID ↔ 테스트 ↔ 커밋으로 추적 가능하게 만든다.



APPENDIX

부록 — 사례 · 이식 · 심화

같은 C2C 구조가 안전(제조 RAG)과 성장(모바일 MVP) 두 도메인에서 어떻게 인스턴스화되는지, 그리고 하네스를 다른 도구로 이식·확장하는 법.

제조 모터 과열 트러블슈팅 RAG (안전 중심)

Mom Test가 드러낸 세 가지 고통

P1 경보 시 여러 PDF를 뒤지느라 초기 대응 10-20분 지연

P2 근거 없는 경험칙 추정에 따른 오진 위험

P3 급박한 상황에서 LOTO 확인이 뒤로 밀리는 안전 절차 누락

ID	계약 (대부분 L0)
INV-2	답변은 반드시 매뉴얼 근거 id/섹션 포함
INV-3	원인/점검/안전/근거 4분할 형식
INV-4	근거 부재 시 '문서상 확인 불가' 강제
E-2	안전 조치(LOTO·보호구) 없는 정비 지시 금지
E-4	실제 설비 제어 기능의 MVP 포함 금지

종단 추적 — 발화 “급하면 LOTO를 나중에 한다” → 고통 P3 → E-2·AC-3 계약화 → 프롬프트 하네스 검사 + 골든 퀘스천 E2E → 하네스 구현 → 안전 문구 포함률 계측으로 귀환. 환각 함정 골든셋(문서에 없어 확신 없던 질문)이 RAG의 거절 유형 원천이 됩니다.

소비자 모바일 그로스 MVP (성장 중심)

정량 목표에서 출발
유입 3,000 · 가입 300 (30일)

job: “30초 안에 나만의 재미있는 결과물을 만들어 SNS에 공유하고 반응을 얻고 싶다”

→ 5개 구현 후보를 ICE로 비교 → AI 스타일 변환 앱 선정

ID	계약 — 성장과 신뢰가 같은 형식으로
INV-1	60초 TTV — 최초 실행 후 60초 내 결과 확인
INV-2	공유 이미지에 앱 설치 딥링크 자동 포함
INV-3	게스트 1회 무료 · 가입 시 워터마크 제거
INV-4	업로드 사진 24시간 내 자동 삭제 (프라이버시)
E-1	변환 실패 시 크레딧을 차감하면 안 된다

AARRR 역산 — 가입 300 → Activation 대비 전환율 약 14.3%, 유입 대비 Activation 70% 가정 → 상위 퍼널 규모 도출. K-factor 0.3으로 바이럴 루프 설계. 성장 계약과 규제 계약(스토어 심사)이 같은 형식으로 공존합니다.

같은 C2C 구조, 도메인별 인스턴스화

구성 요소	사례 1 — 모터 과열 RAG (안전)	사례 2 — 그로스 MVP (성장)
Layer 1 불변식	근거 인용 강제·확인 불가 응답·안전 조치	60초 TTV·공유 답링크·24h 사진 삭제
Layer 2 UI 계약	역할별 원문 마스킹·4분할 답변	크레딧 소진 시 가입 유도·실패 재시도
Revenue 재정의	비용 절감·안전 리스크 저감	가입 전환
접근 게이트	RBAC 4역할 × 6권한	AccessGate 2상태 × 4기능
품질 지표	groundedness 100%·환각률 0%	TTV ≤ 60초·K-factor ≥ 0.3
테스트 하네스	FakeEmbedder/FakeStore/FakeLLM	Fake Auth/StyleApi/Analytics

도메인 특성은 체계의 구조를 바꾸지 않고 세 지점에서 파라미터로 흡수됩니다 — ① 계약의 내용 ② AARRR의 재정의 ③ 하네스의 Fake 대상. 13단계와 C2C 3요소는 두 도메인에서 누락 없이 동형적으로 유지됩니다.

Codex · Claude Code 협업 — 하네스는 이식된다

4장 하네스(지도 + 설정 + 스킴 + 금지 + 계약 ID)는 도구 기능이 아니라 “규율”입니다. 파일 이름과 설정 키만 바뀔 뿐, 계약·테스트·금지는 그대로 이식됩니다 .

	① Codex CLI (본 과정)	② Claude Code
지도 파일	AGENTS.md	CLAUDE.md
설정 파일	~/.codex/config.toml · .codex/config.toml	.claude/settings.json
계약·테스트·금지	docs/PRD.md · pytest -q — 두 도구 동일	

두 도구가 함께 읽게 하는 법 — Codex는 AGENTS.md 기본 인식 · Claude Code는 CLAUDE.md에서 @AGENTS.md 참조 또는 심링크(ln -s AGENTS.md CLAUDE.md). 승인·샌드박스 기본값은 도구마다 다르니 보수적으로 시작(read-only · plan).

AI 시대 개발자 역할의 재정의 — 그리고 한계

비교우위의 이동

AI가 계약→구현 비용을 낮춘 상황에서 인간의 비교우위는:

- ① 상류 — 불변식을 발견·계약화한다
- ② 하류 — 계측을 설계해 개념의 실증 여부를 판정한다
- ③ 사슬 전체의 무단절성 감리

→ 코드의 작성자에서 C2C 추적성의 설계자·감리자로 재정의

한계와 타당성 위협

- 통제 실험이 아닌 설계 과학적 시연 · 질적 교차 사례
- 연구자가 설계에 참여한 실행 연구 — 제3자 재현 필요
- AARRR 역산의 전환율 가정은 도메인·시점에 따라 변동
- 향후 — 계약에서 테스트·계측을 자동 생성하는 C2C 도구 체인

본 체계는 새 기법의 발명이 아니라 검증된 기법들의 목적론적 재조직입니다 — 기여는 이들을 C2C라는 단일 목표 아래 배치하고, 그 변환 규칙을 계약 ID와 매트릭스로 형식화한 데 있습니다.

MCP — 외부 도구 연결 (선택 심화)

```
# ~/.codex/config.toml (또는 .codex/config.toml)
[mcp_servers.sequential-thinking]
command = "npx"
args = ["-y", "@modelcontextprotocol/server-sequential-thinking"]
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp", "--api-key", "KEY"]
[mcp_servers.github]
url = "https://api.githubcopilot.com/mcp/"
```

무엇이 되나

context7 — 프롬프트 끝에 use context7 → 버전에 맞는 최신 라이브러리 문서를 찾아온다.

github — Fine-grained PAT 발급 후 “내 저장소 목록”·“이슈 생성” 같은 작업.

MCP 서버 역시 하네스의 일부 — 어떤 도구가 연결됐는지 /mcp로 항상 파악하고, 신뢰할 수 없는 서버는 연결하지 않습니다. 변경 후에는 세션을 재시작합니다.

훅(Hooks) — 감사 로그 자동 기록

PostToolUse 훅

```
# .codex/config.toml
[[hooks.PostToolUse]]
matcher = ".*"
[[hooks.PostToolUse.hooks]]
type = "command"
command = ".codex/hooks/audit-log.sh"

# audit-log.sh — 실행마다 한 줄 (stdin = 훅 JSON)
echo "$(date) | $(jq -r .tool_name)" >> docs/audit.log
```

chmod +x — 실행권한 (필수)

실행권한이 없으면 훅은 조용히 실패합니다.

```
chmod +x .codex/hooks/audit-log.sh
```

ls -l 로 +x 확인.

git log가 “무엇을 커밋했는가”라면, 감사 로그는 “에이전트가 무엇을 실행했는가” — 추적 사슬의 런타임 증거.

훅은 신뢰할 수 있는 스크립트만 등록하고, 로그에 토큰·비밀번호를 남기지 않습니다.



CLOSING

“이 코드 한 줄은 어느 계약을 지키려는 것인가?”

이 질문에 즉시 답할 수 있게 하는 것 — 그것이 C2C입니다. 여러분은 이제 코드의 작성자가 아니라, AI와 협업하며 C2C 추적 사슬을 설계·감리하는 사람입니다.

■ 계약 ID 없는 동작은 구현하지 않는다

■ 실패 로그가 최고의 컨텍스트다

■ 테스트 약화는 즉시 거절한다

■ 확장 뒤에는 수축을 둔다

■ 구조 변경과 동작 변경을 섞지 않는다

■ 매트릭스의 빈칸이 다음 질문이다